Graded Assignment

Travel Memory Application Deployment on AWS

MERN stack on Amazon EC2 with Nginx, Application Load Balancers, Auto Scaling, Terraform and a Cloudflare custom domain

Student	Ibrahima Samake
Programme	PPMCAD – Multi Cloud Architecture & DevOps (Hero Vired)
Live application	https://travelmemory.samibrahima.tech
AWS region	ap-south-1 (Asia Pacific – Mumbai)
Source code (GitHub)	https://github.com/ibrahimasamake/travel-memory-aws-deployment
Date	25 October 2026

Contents

1. Introduction and objectives

2. Requirement checklist

3. Deployment architecture

4. Phase 1 – Backend configuration and Nginx reverse proxy (Task 1)

5. Phase 1 – Frontend and backend connection (Task 2)

6. Scaling the application (Task 3)

7. Custom domain with Cloudflare (Task 4)

8. Security, scalability and resilience

9. Issues met and how they were solved

10. Command reference

11. Conclusion and submission

1. Introduction and objectives

Travel Memory is a MERN application (MongoDB, Express, React, Node.js) that lets users record travel experiences. The codebase comes from https://github.com/UnpredictablePrashant/TravelMemory and has two parts: a Node.js/Express API in **backend/** and a React single-page application in **frontend/**.

This document describes, step by step and with screenshots, how the application was deployed on Amazon EC2. The work was done in two phases:

- **Phase 1 (manual, AWS console):** one frontend EC2 instance in a public subnet and one backend EC2 instance in a private subnet. Both run under pm2, and Nginx acts as the reverse proxy. These instances were then turned into AMIs, launch templates and Auto Scaling groups.
- **Phase 2 (Infrastructure as Code, Terraform):** the network (VPC, subnets, Internet Gateway, NAT Gateway, route tables), the launch templates, an internet-facing ALB, an internal ALB and two Auto Scaling groups are all defined in Terraform. HTTPS uses an ACM certificate, and the custom domain is served through Cloudflare DNS.

Objectives covered:

- Set up the backend running on Node.js.
- Configure the React frontend.
- Make the frontend and backend communicate reliably.
- Deploy the full application on EC2.
- Run multiple instances behind load balancers.
- Connect a custom domain through Cloudflare.

2. Requirement checklist

The table maps each requirement in the assignment brief to the section and figures that show it.

Requirement	How it was implemented	Evidence
1. Clone repo, go to backend	git clone on the EC2 instance, cd TravelMemory/backend	§4.2, Fig. 4
1. Backend port + Nginx reverse proxy	Express API under pm2; Nginx on :80 proxies /trip to the API and / to React	§4.3–4.5, Fig. 5–7
1. Update .env (DB + port)	backend/.env holds MONGO_URI and PORT=3001	§4.3
2. Update urls.js in frontend	frontend/src/url.js points baseUrl to the frontend, then the load balancer, then the custom domain	§5.1, §6.7, §7.5, Fig. 24, 30
3. Multiple frontend and backend instances	AMIs, launch templates, 2 ASGs (min 2 / max 4), first in the console and then in Terraform	§6, Fig. 12–23
3. Load balancer	Internet-facing my-app-alb for the frontend; internal my-app-backend-alb for the API	§6.5, §7.2, Fig. 21, 28
4. Cloudflare CNAME → load balancer	travelmemory.samibrahima.tech CNAME → my-app-alb DNS name	§7.3, Fig. 29
4. Cloudflare A record → EC2 IP	A record → public IP of a frontend EC2 instance	§7.4
5. Documentation + draw.io diagram	This document; TravelMemory-Architecture.drawio	§3, Fig. 1

3. Deployment architecture

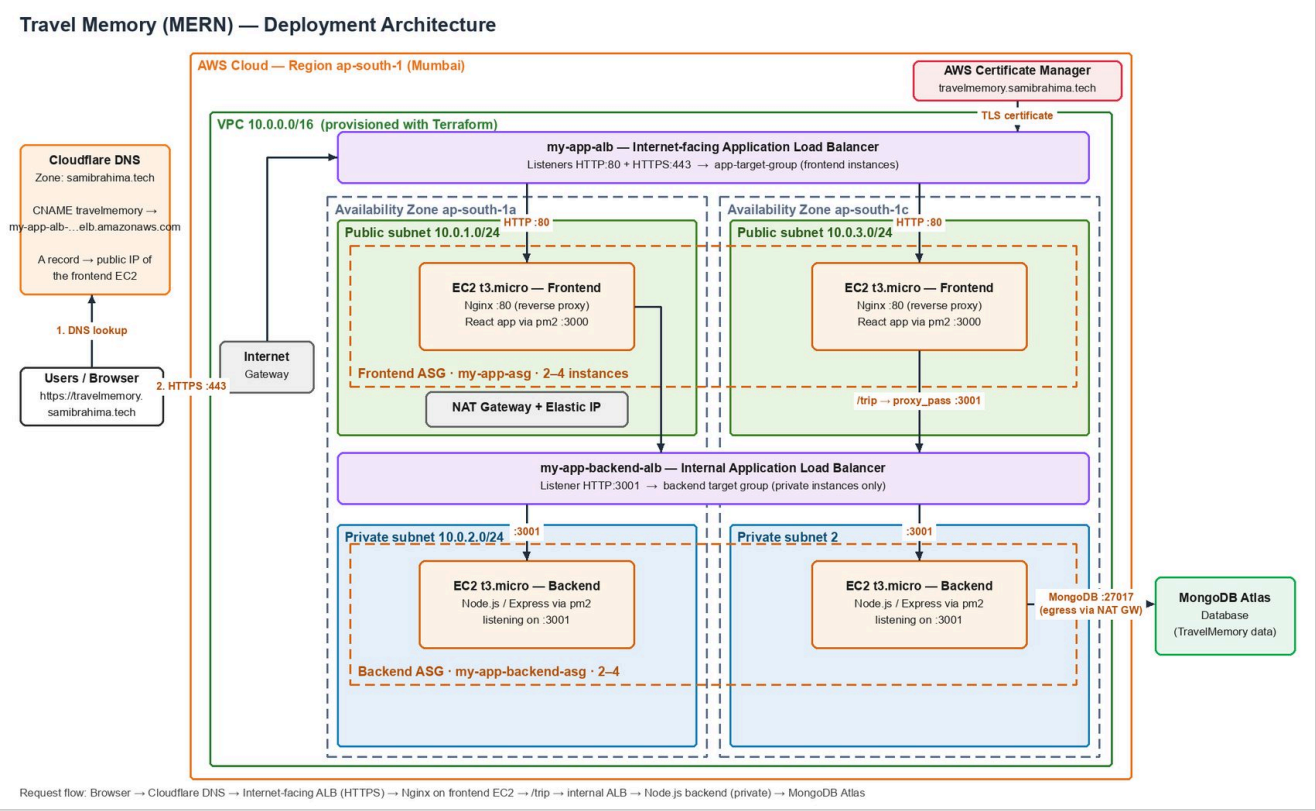


Figure 1 – Deployment architecture (editable source: TravelMemory-Architecture.drawio)

Request flow

1. The browser resolves travelmemory.samibrahima.tech through Cloudflare DNS (CNAME → my-app-alb).
2. The internet-facing Application Load Balancer terminates HTTPS on port 443 with an ACM certificate and forwards to app-target-group, which holds the frontend instances.
3. On each frontend instance, Nginx listens on port 80. It serves the React app (proxy to localhost:3000) and forwards every /trip API call to the internal backend ALB on port 3001.
4. The internal ALB spreads API calls over the backend instances in the private subnets. Express listens on port 3001.
5. The backend reads and writes MongoDB Atlas. Outbound traffic from the private subnets leaves through the NAT Gateway.

Component	Details
VPC	10.0.0.0/16, DNS support and DNS hostnames enabled (Terraform aws_vpc.main)
Public subnets	10.0.1.0/24 (ap-south-1a) and 10.0.3.0/24 (second AZ): ALB nodes, frontend EC2 instances, NAT Gateway
Private subnets	10.0.2.0/24 (ap-south-1a) and private subnet 2 (ap-south-1c): backend EC2 instances, no public IP
Internet / NAT Gateway	IGW for public subnets; NAT Gateway with an Elastic IP for private-subnet egress (npm, MongoDB Atlas)
my-app-alb	Internet-facing ALB, listeners HTTP:80 and HTTPS:443 (policy ELBSecurityPolicy-TLS13-1-2) → app-target-group
my-app-backend-alb	Internal ALB, listener HTTP:3001 → backend target group
Auto Scaling groups	my-app-asg (frontend) and my-app-backend-asg (backend): min 2, desired 2, max 4, t3.micro
DNS / TLS	Cloudflare zone samibrahima.tech; ACM certificate for travelmemory.samibrahima.tech
Database	MongoDB Atlas (connection string in backend/.env)

4. Phase 1 – Backend configuration and Nginx reverse proxy (Task 1)

4.1 Launch the EC2 instances

Two Ubuntu 26.04 instances (t2.micro) were launched in the Hero-Vpc network: **Travel-Agency-Frontend** in a public subnet with public IP 13.201.121.187, and **Travel-Agence-Backend** in a private subnet (private IP 10.0.1.9, no public IP). Keeping the backend in a private subnet means the API is never directly reachable from the internet.

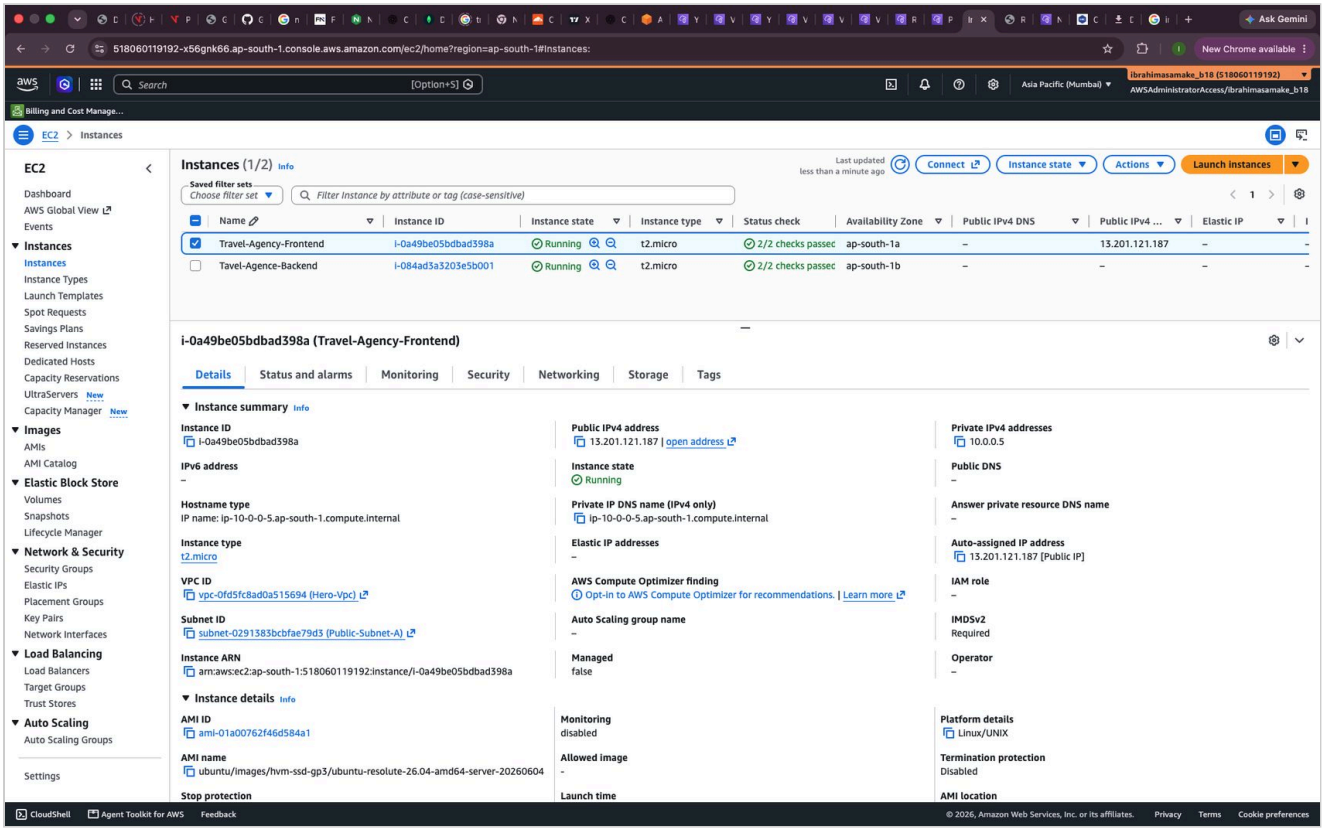


Figure 2 – Frontend instance – public subnet, public IP 13.201.121.187

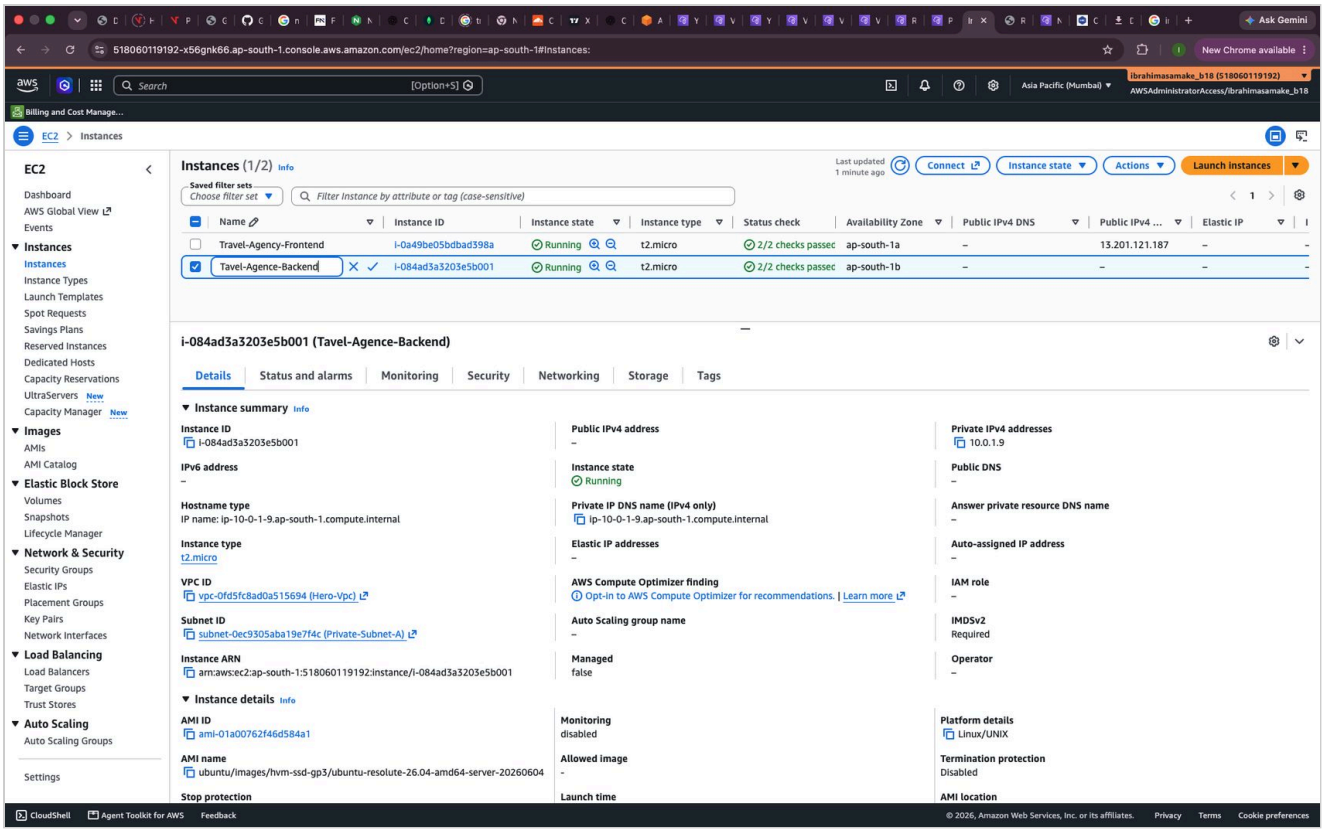


Figure 3 – Backend instance – private subnet (Private-Subnet-A), private IP only

4.2 Install packages and clone the repository

On each instance, update the system, install Node.js, npm and git, then clone the project and open the backend folder:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y nodejs npm git nginx
git clone https://github.com/UnpredictablePrashant/TravelMemory.git
cd TravelMemory/backend
npm install
```

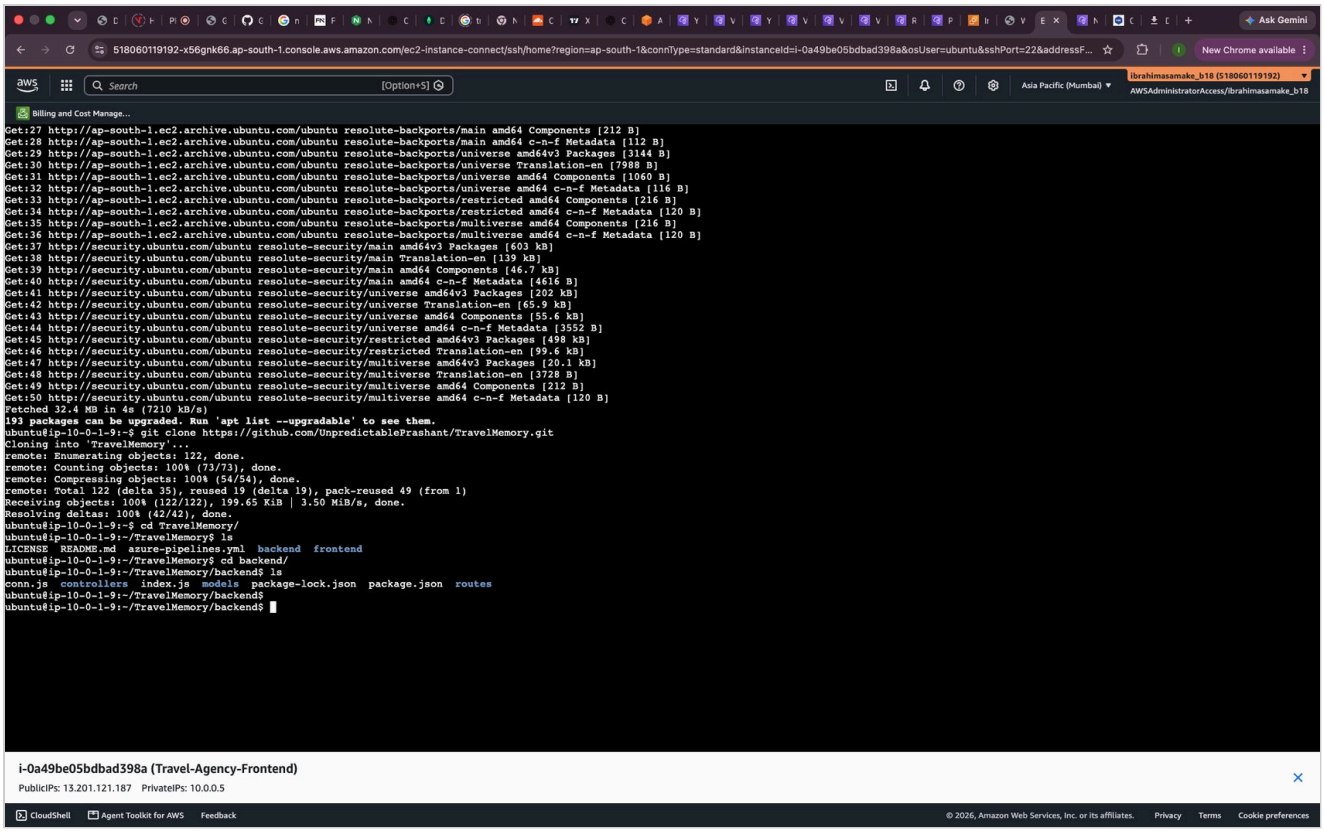


Figure 4 – Repository cloned; backend folder contains index.js, conn.js, controllers, models, routes

4.3 Configure the .env file (database + port)

The backend reads its settings from **backend/.env**. The React development server already uses port 3000 on the frontend instance, so the API was set to **port 3001**. This keeps the two services from clashing when they run on the same host.

```
# TravelMemory/backend/.env
MONGO_URI=mongodb+srv://<db_user>:<db_password>@<cluster-host>/travelmemory?retryWrites=true&w=majority
PORT=3001
```

Add screenshot: `cat backend/.env` with the password masked (e.g. mongodb+srv://user:**@...).**

4.4 Run the backend with pm2

pm2 keeps the Node.js process running, restarts it if it crashes, and starts it again after a reboot. The repository's entry point is **index.js**, not server.js (see §9).

```
sudo npm install -g pm2
pm2 start index.js --name my-backend
pm2 logs my-backend          # -> Server started at http://localhost:3001
pm2 save && pm2 startup      # restart automatically on reboot
```

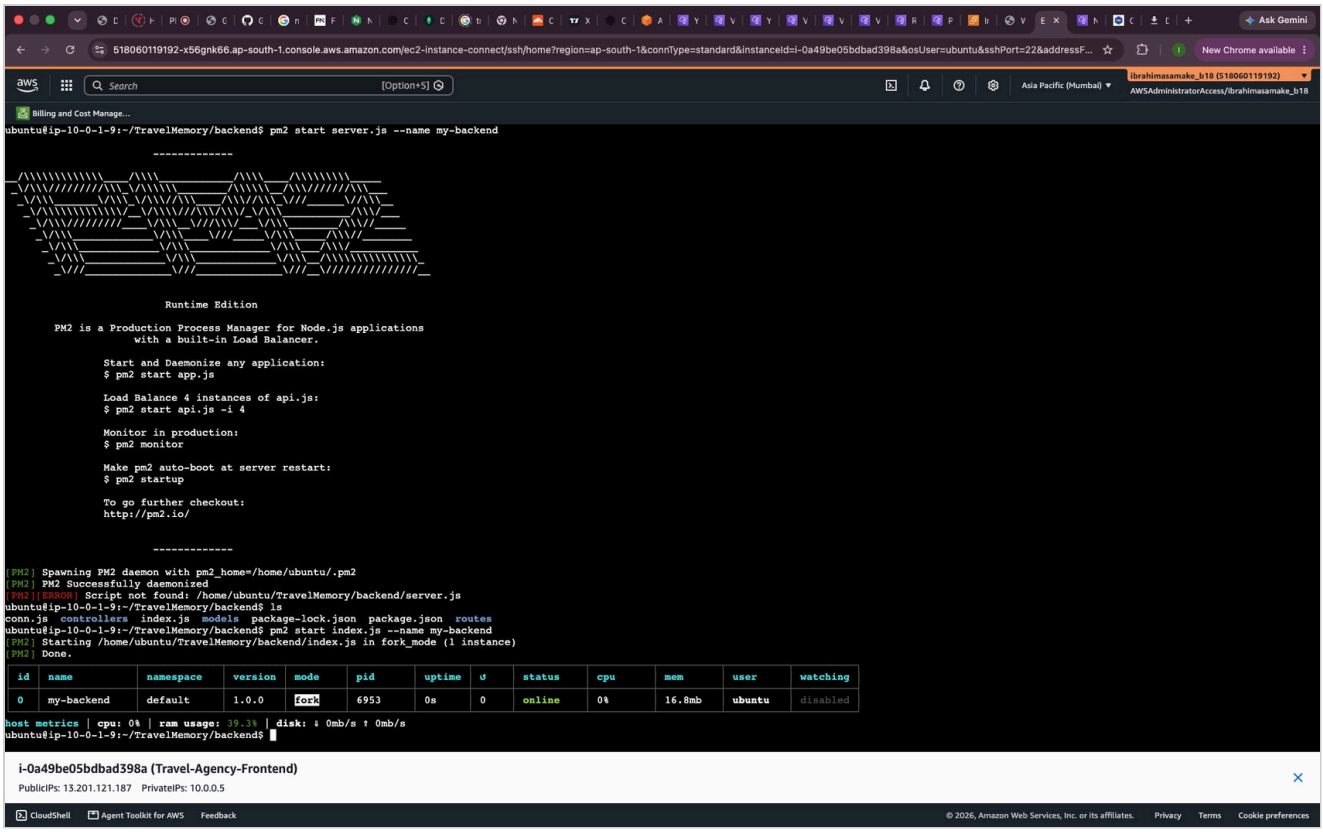


Figure 5 – pm2 installed and backend started (pm2 start index.js --name my-backend) – status online

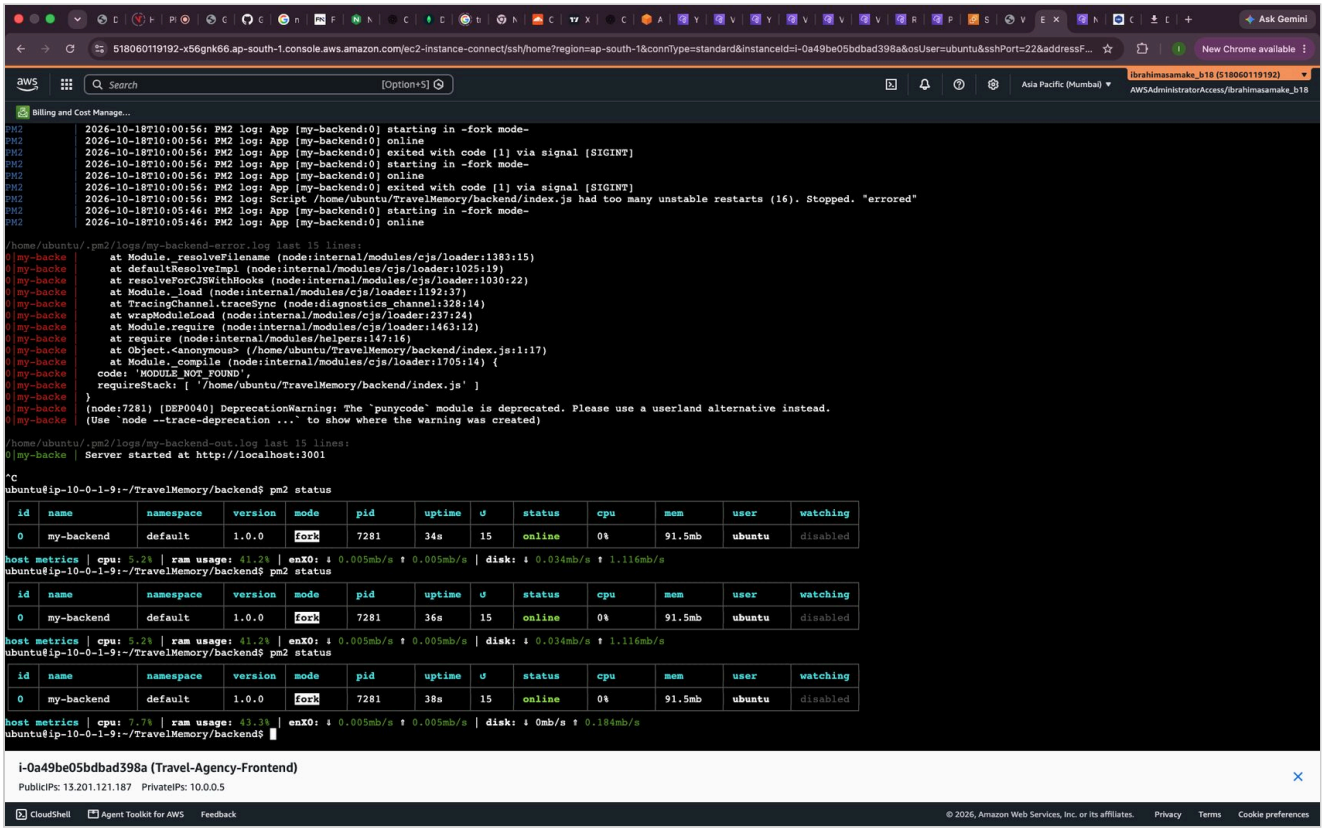


Figure 6 – pm2 logs: 'Server started at http://localhost:3001'; my-backend stays online

4.5 Nginx reverse proxy

On the frontend instance, Nginx listens on port 80 and is the only entry point. Requests to **/trip** (the API path) go to the backend on port 3001. Every other path goes to the React app on port 3000. The

configuration lives in /etc/nginx/sites-available/default:

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;

    location /trip {
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_pass http://10.0.1.9:3001;      # backend private IP (Phase 1)
    }

    location / {
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_pass http://localhost:3000;     # React app
    }
}
```

```
sudo nginx -t
sudo systemctl restart nginx
sudo systemctl enable nginx
```

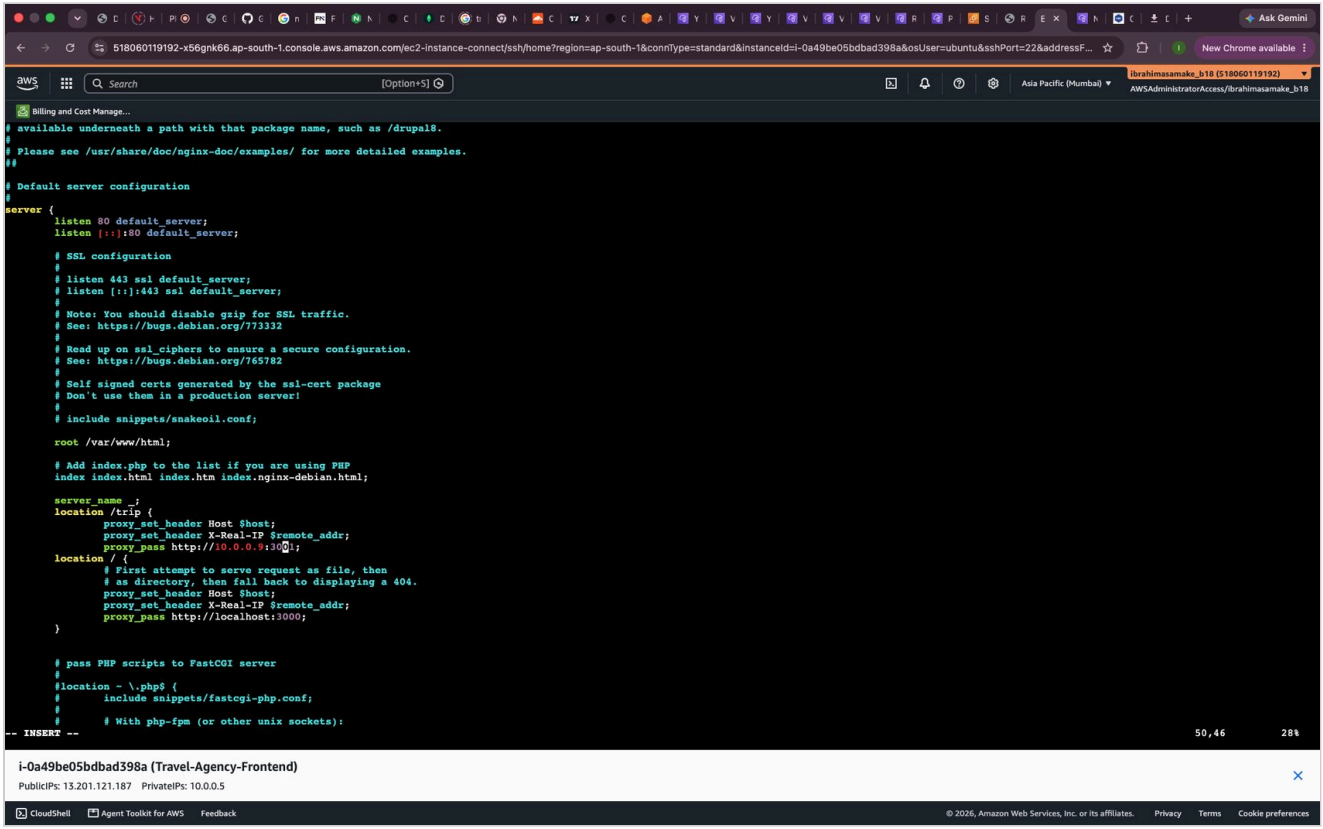


Figure 7 – Nginx default site being edited: /trip proxied to the backend on :3001, / proxied to React on :3000

5. Phase 1 – Frontend and backend connection (Task 2)

5.1 Update url.js

The React app sends every API call to **baseUrl**, which is exported from frontend/src/url.js. Because Nginx already proxies /trip, baseUrl only has to point at the public entry point. In Phase 1 that was the frontend instance; it was later changed to the load balancer (§6.7) and then to the custom domain (§7.5):

```
// TravelMemory/frontend/src/url.js
export const baseUrl = process.env.REACT_APP_BACKEND_URL || "http://13.201.121.187";
```

5.2 Start the frontend

```
cd ~/TravelMemory/frontend
npm install
pm2 start npm --name frontend -- start     # React on :3000
pm2 save
pm2 startup
```

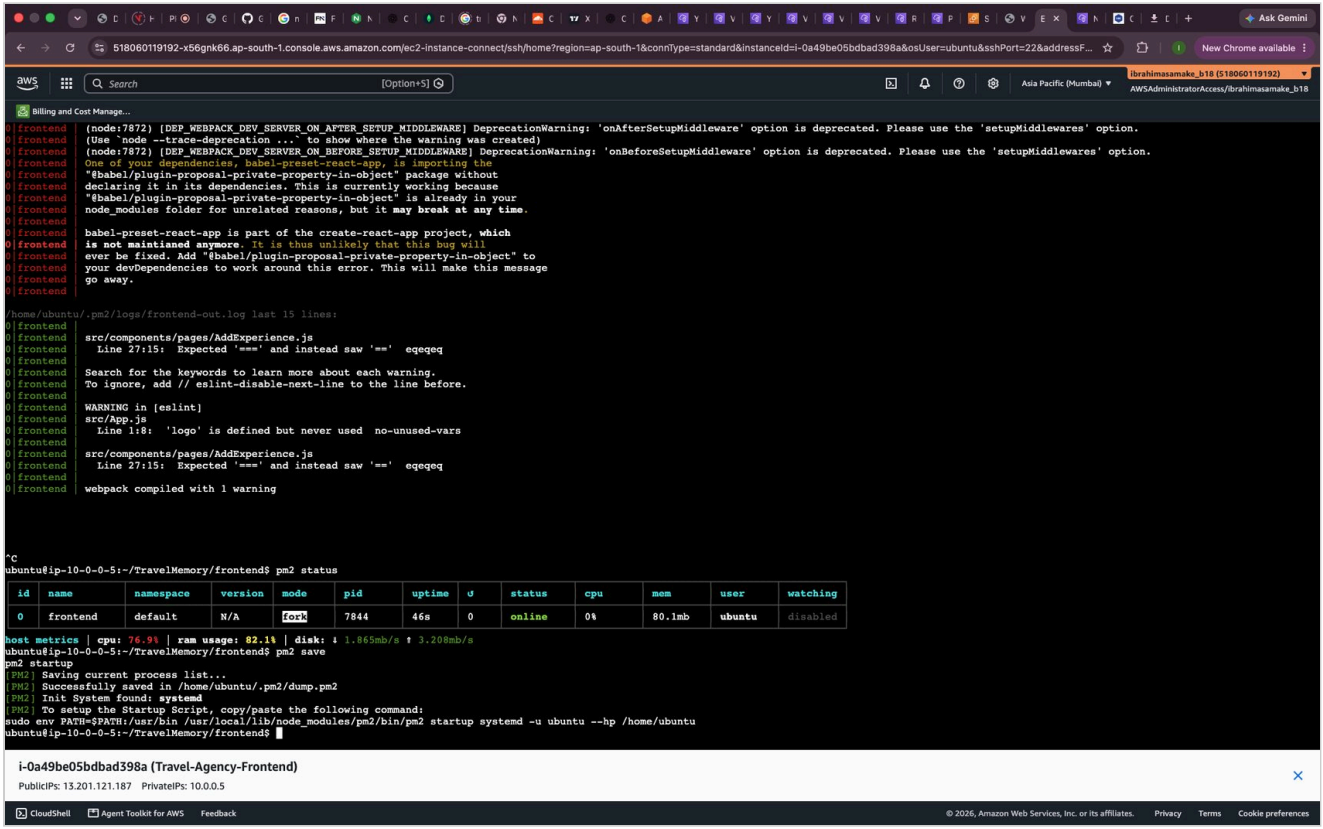


Figure 8 – Frontend compiled and running under pm2 (status online); pm2 save / pm2 startup configured

5.3 Verification

Opening http://13.201.121.187 loads the app. In the browser's Network tab, the **trip/** XHR request returns **200**. The response headers show **Server: nginx/1.28.3** and **X-Powered-By: Express**, which proves the request went through Nginx to the Express backend. Opening a record's detail page confirms that data is read from MongoDB.

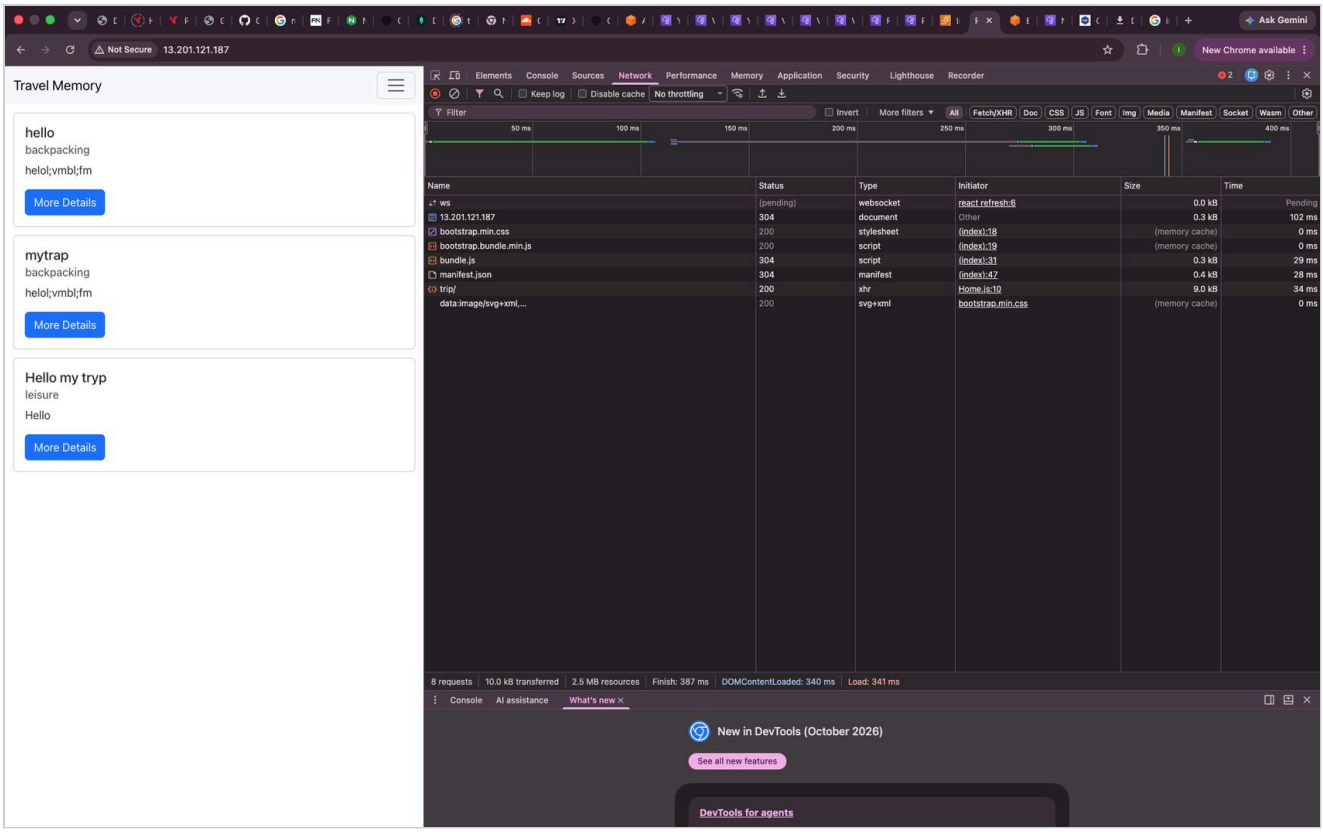


Figure 9 – App served on the frontend public IP – trip/ API call returns 200

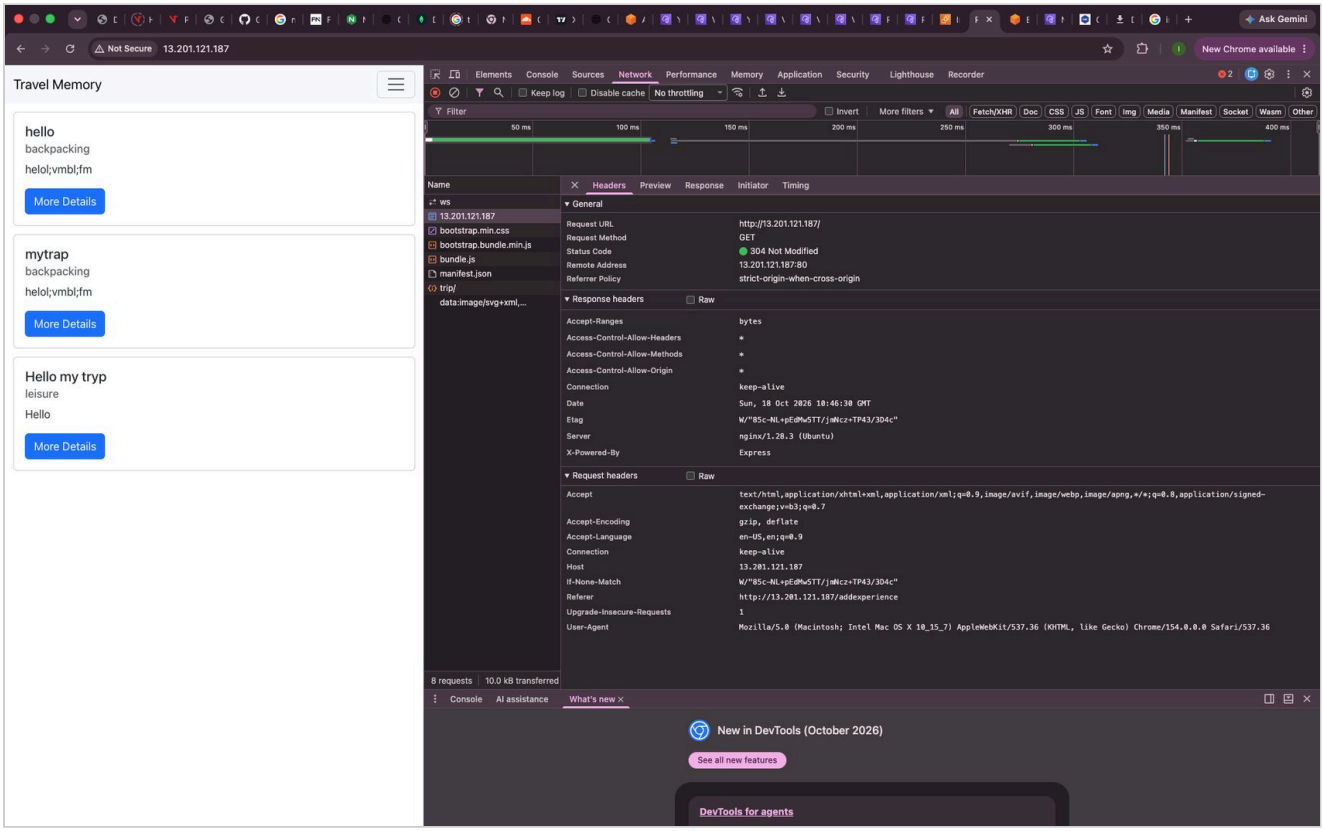


Figure 10 – Response headers: nginx (reverse proxy) + Express (backend)

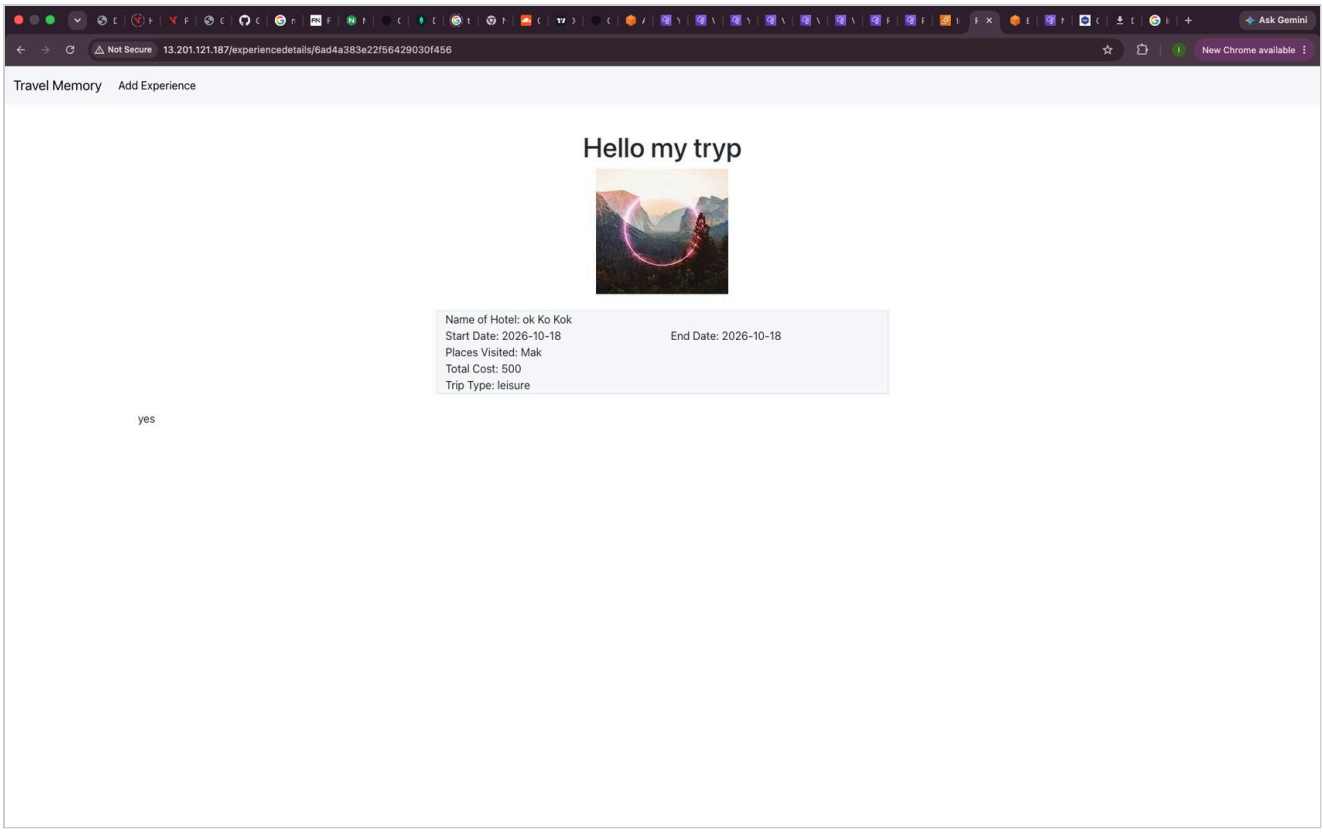


Figure 11 – Experience details page – data loaded from the database

6. Scaling the application (Task 3)

6.1 Create AMIs from the configured instances

Once both instances worked, an AMI was created from each one (EC2 → Instances → Actions → Image and templates → Create image). Every new instance starts from these images with Node.js, the code, pm2 and Nginx already set up.

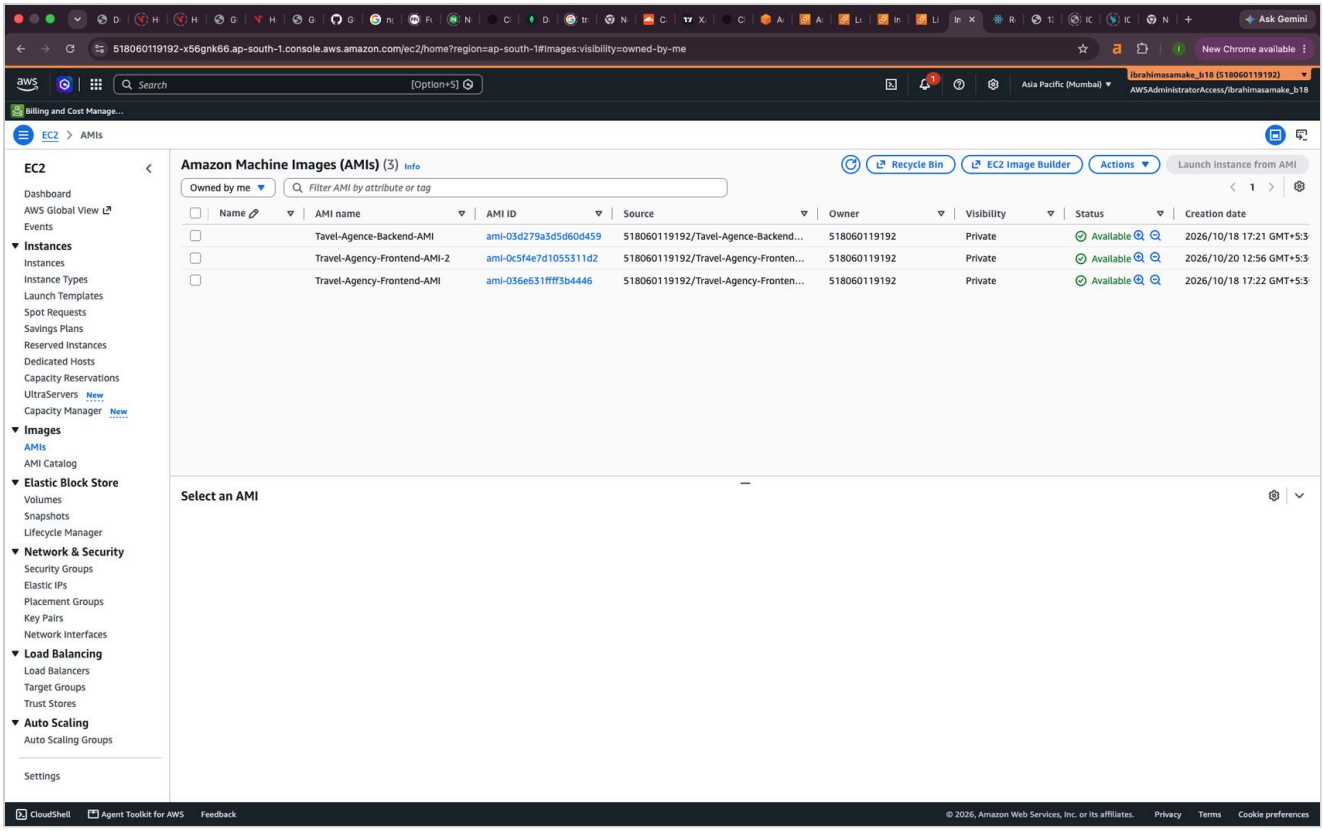


Figure 12 – AMIs: Travel-Agence-Backend-AMI, Travel-Agency-Frontend-AMI and Travel-Agency-Frontend-AMI-2

6.2 Launch templates

Launch templates hold the AMI, instance type, key pair and security group used for each tier. The Travel-Agency-* templates were built by hand in the console. The app-template-my-* templates were created by Terraform in Phase 2.

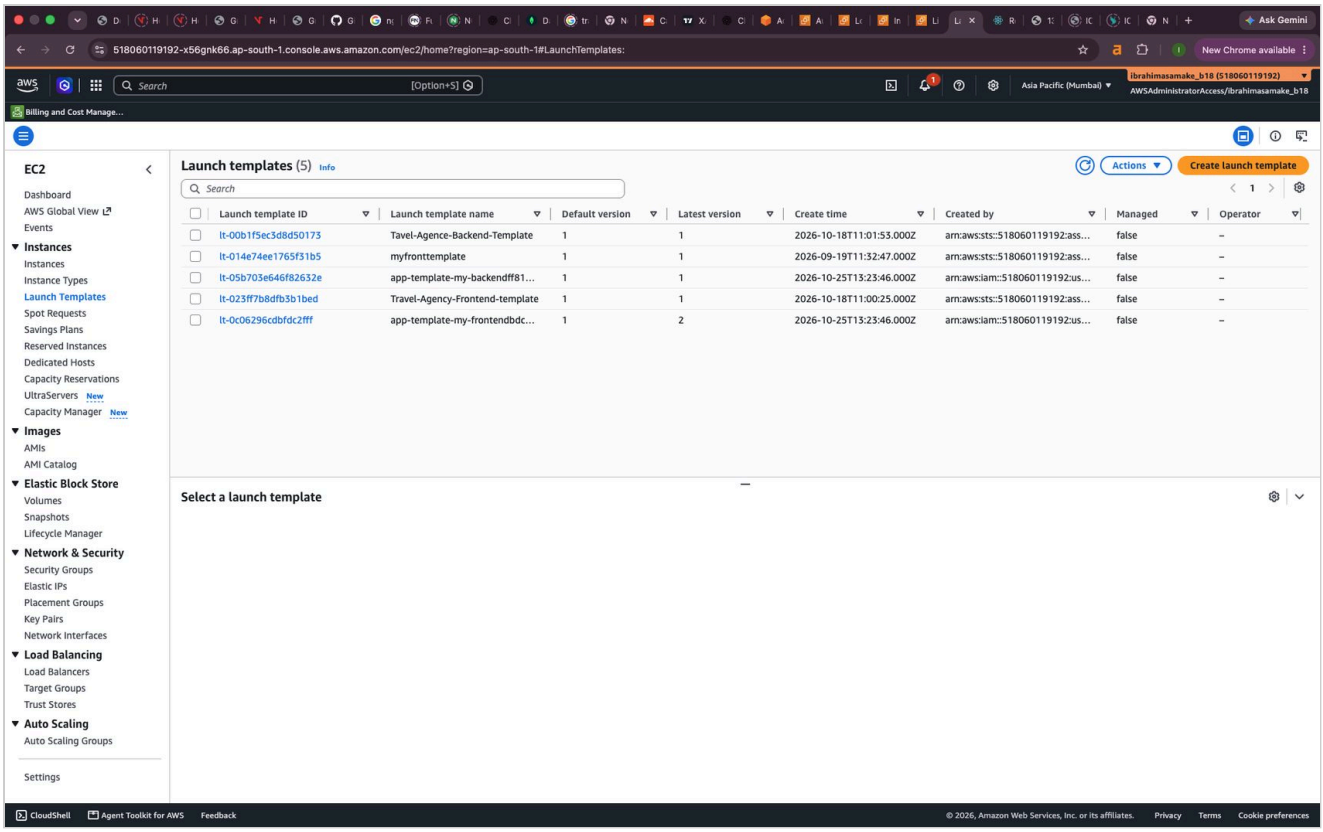


Figure 13 – Launch templates for the frontend and backend tiers

6.3 First Auto Scaling groups (console)

Two Auto Scaling groups were created from the launch templates (desired 2, min 2, max 4). Within a few minutes the account showed three frontend and three backend instances running.

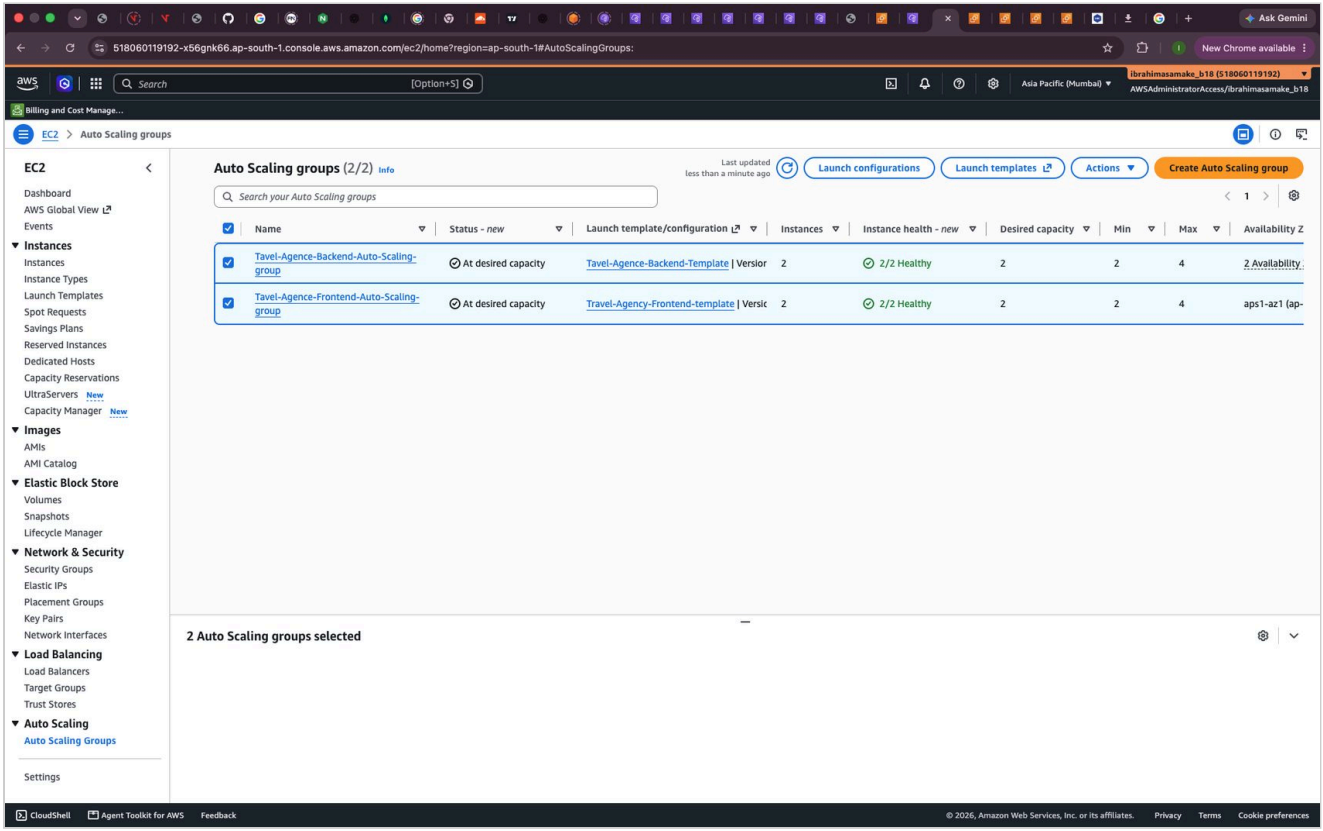


Figure 14 – Console-built ASGs: Travel-Agence-Backend and Travel-Agency-Frontend, 2/2 healthy each

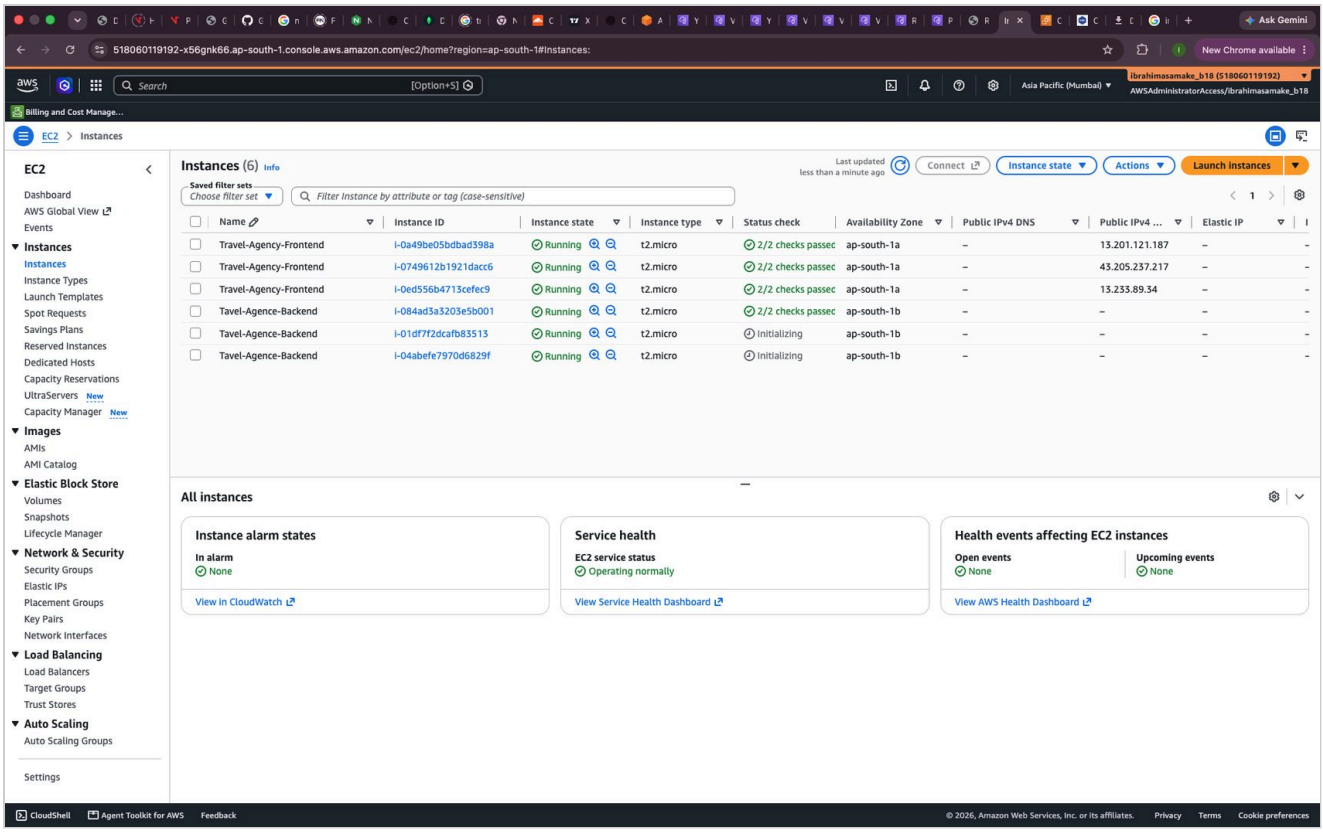


Figure 15 – Multiple frontend and backend instances running

6.4 Rebuilding the infrastructure with Terraform

To make the deployment reproducible, the whole stack was rewritten in Terraform (folder Vpc-in-aws: main.tf, variables.tf, outputs.tf, terraform.tfvars). It is deployed with:

```
terraform init
terraform fmt && terraform validate
terraform plan –out tfplan
terraform apply tfplan
terraform output
```

main.tf declares the AWS provider (hashicorp/aws ~> 6.0), the VPC, public and private subnets in two Availability Zones, the Internet Gateway, the NAT Gateway with its Elastic IP, route tables, security groups, both load balancers, target groups, launch templates and Auto Scaling groups.

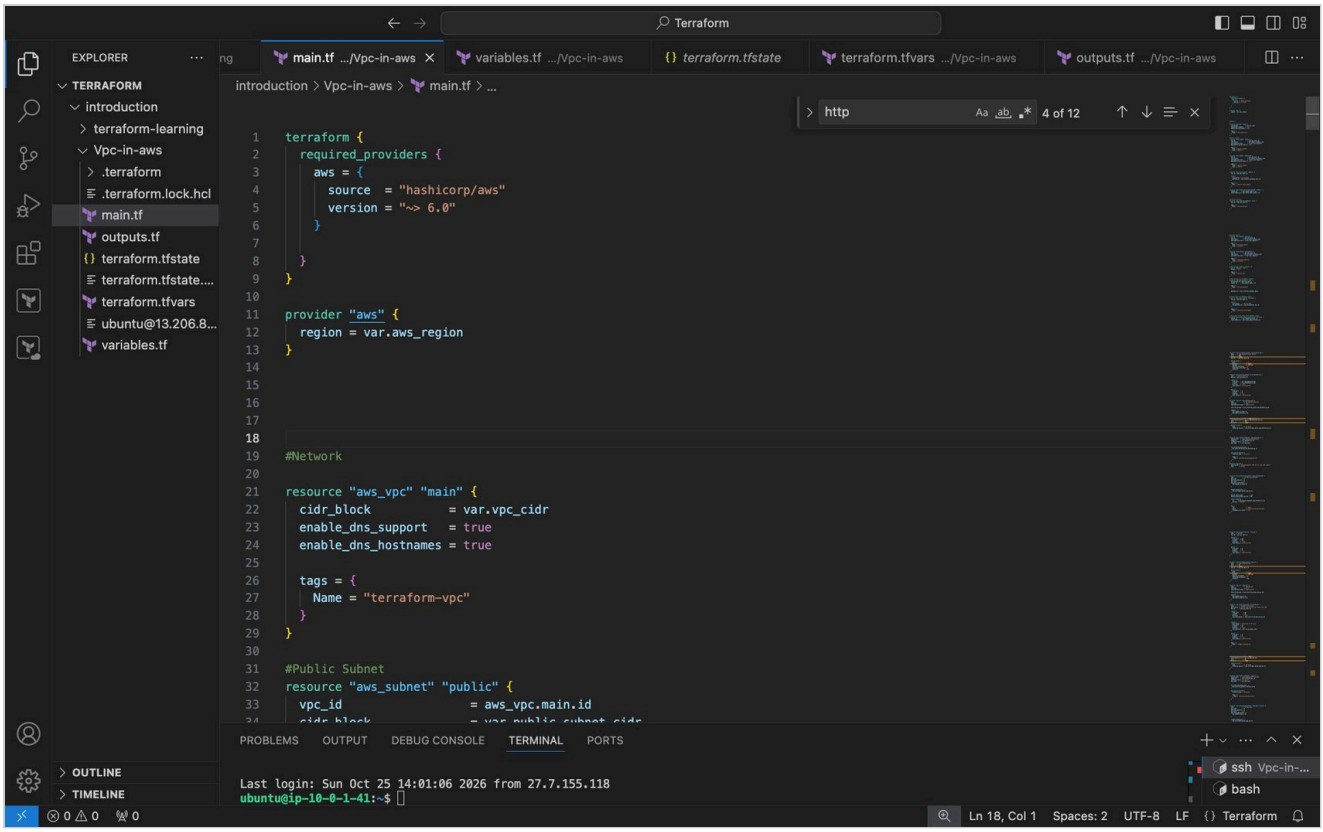


Figure 16 – main.tf – provider and VPC 10.0.0.0/16

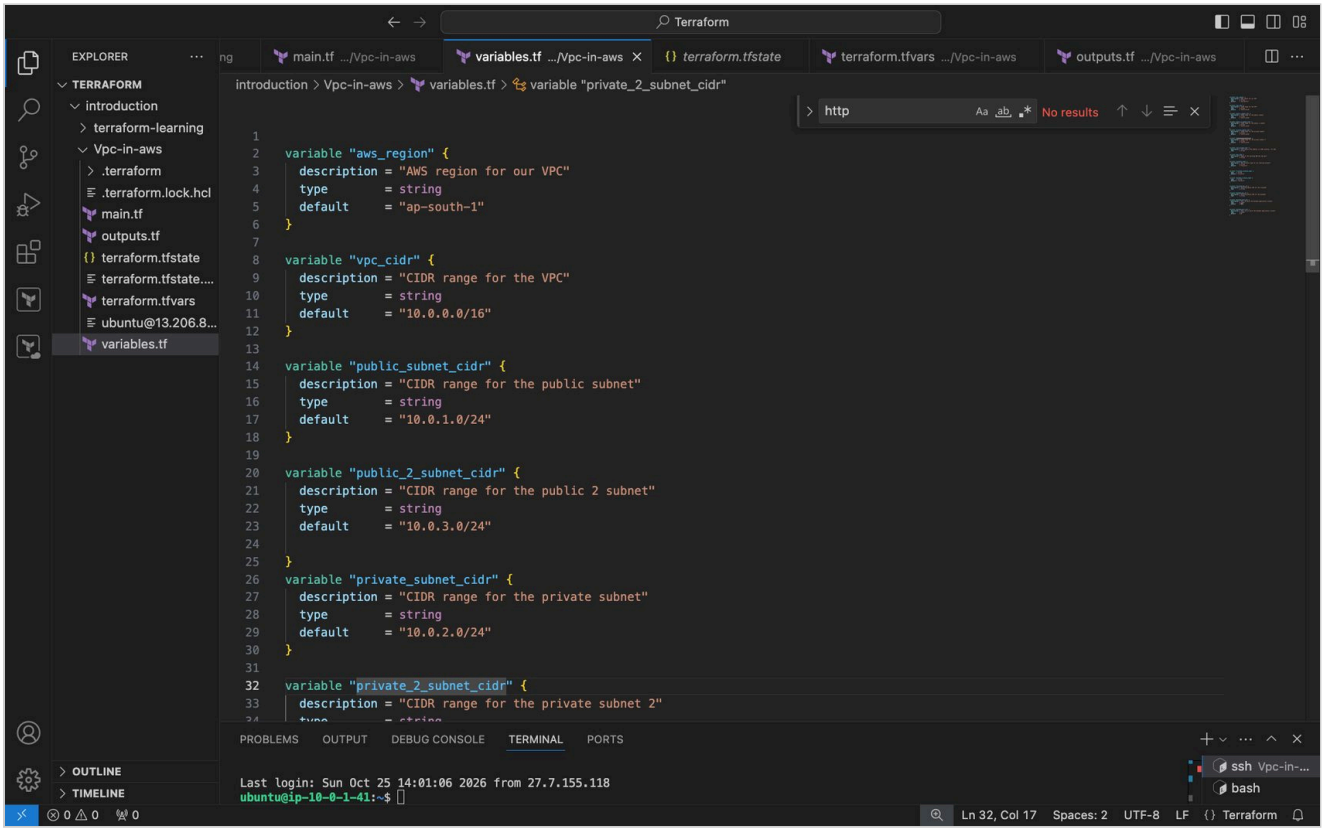


Figure 17 – variables.tf – region ap-south-1 and the subnet CIDR blocks

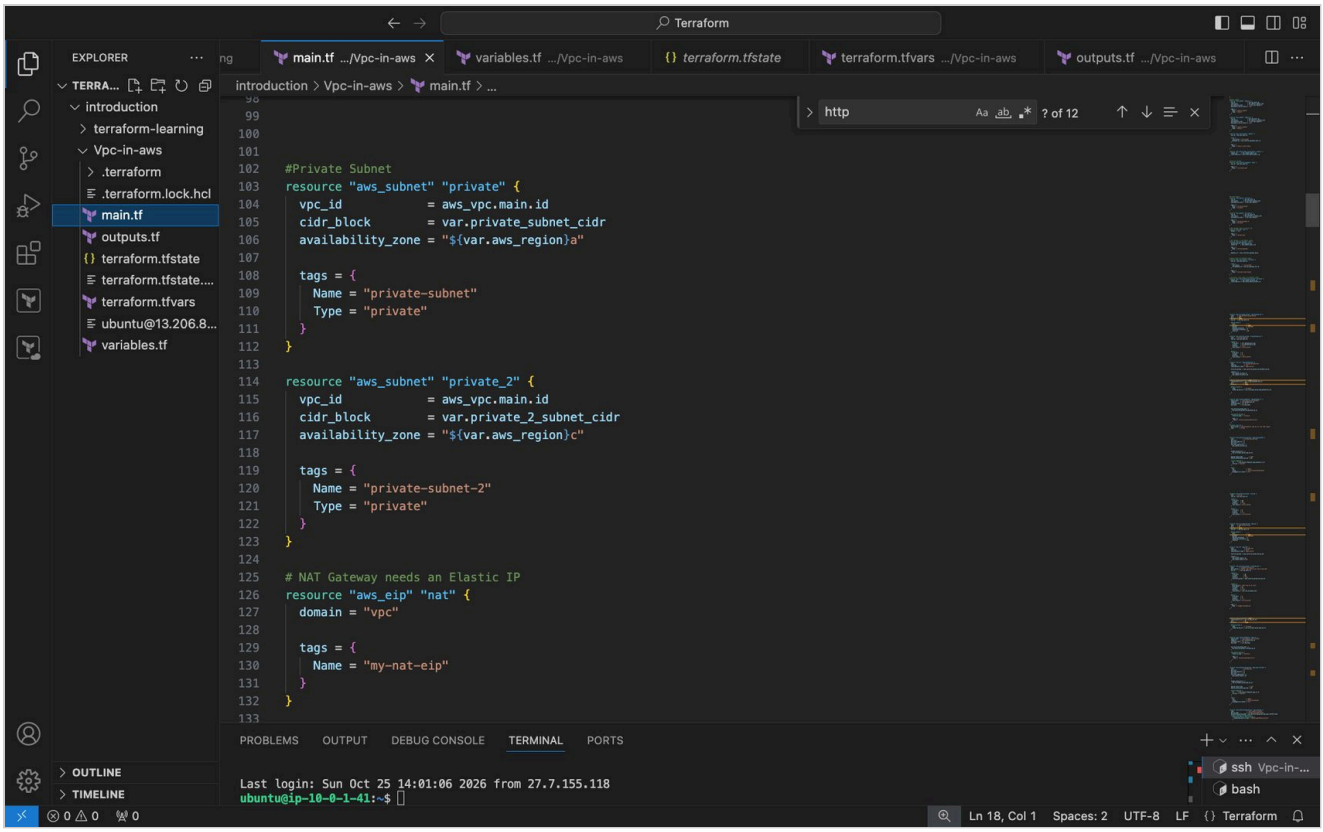


Figure 18 – main.tf – private subnets in two AZs and the Elastic IP for the NAT Gateway

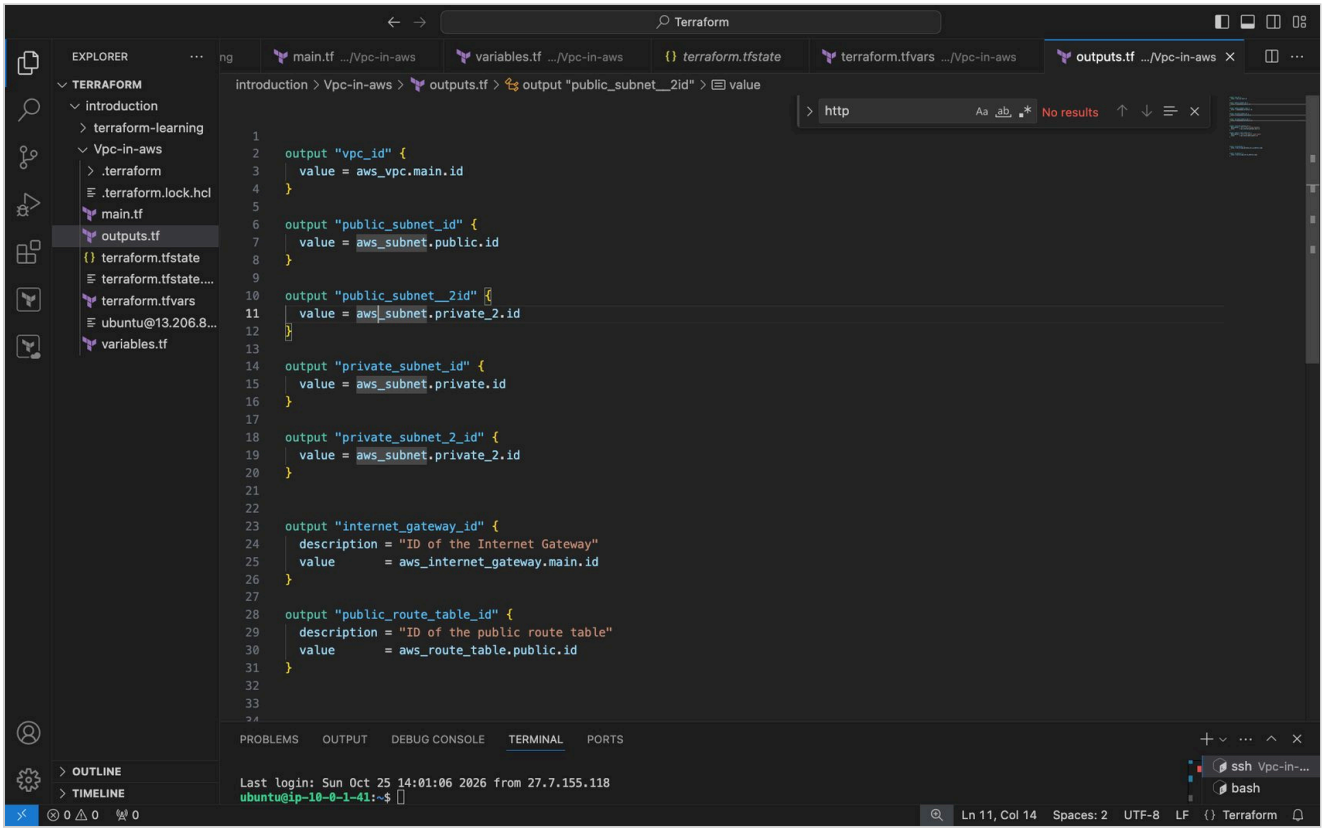


Figure 19 – outputs.tf – VPC, subnet, IGW and route-table IDs

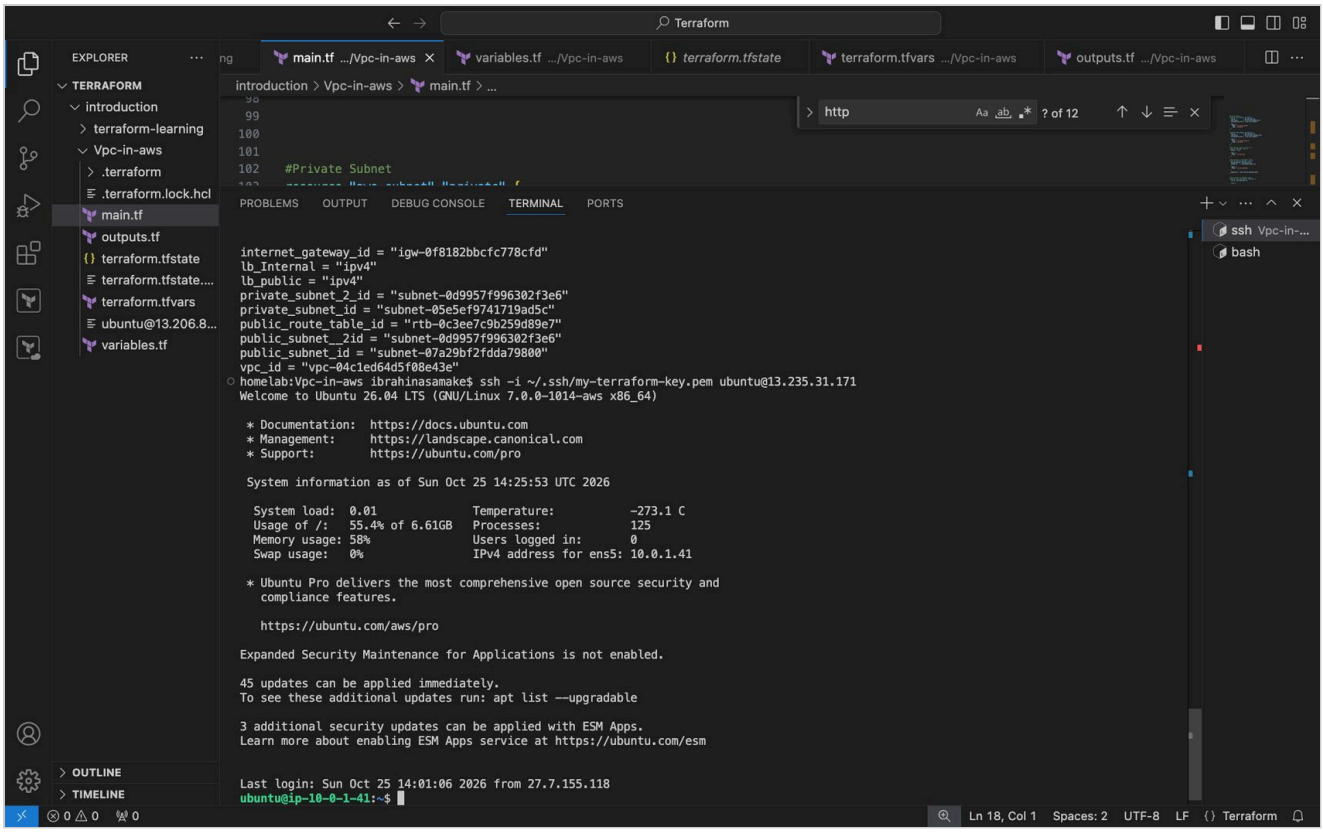


Figure 20 – terraform output and SSH into a frontend instance launched by the ASG (13.235.31.171)

6.5 Load balancers

Two Application Load Balancers sit in the same VPC (vpc-04c1ed64d5f08e43e) across two Availability Zones:

- **my-app-alb** – internet-facing. It receives user traffic and forwards it to the frontend target group.
- **my-app-backend-alb** – internal, with no public DNS resolution. It is reachable only from inside the VPC and spreads API calls over the backend instances on port 3001.

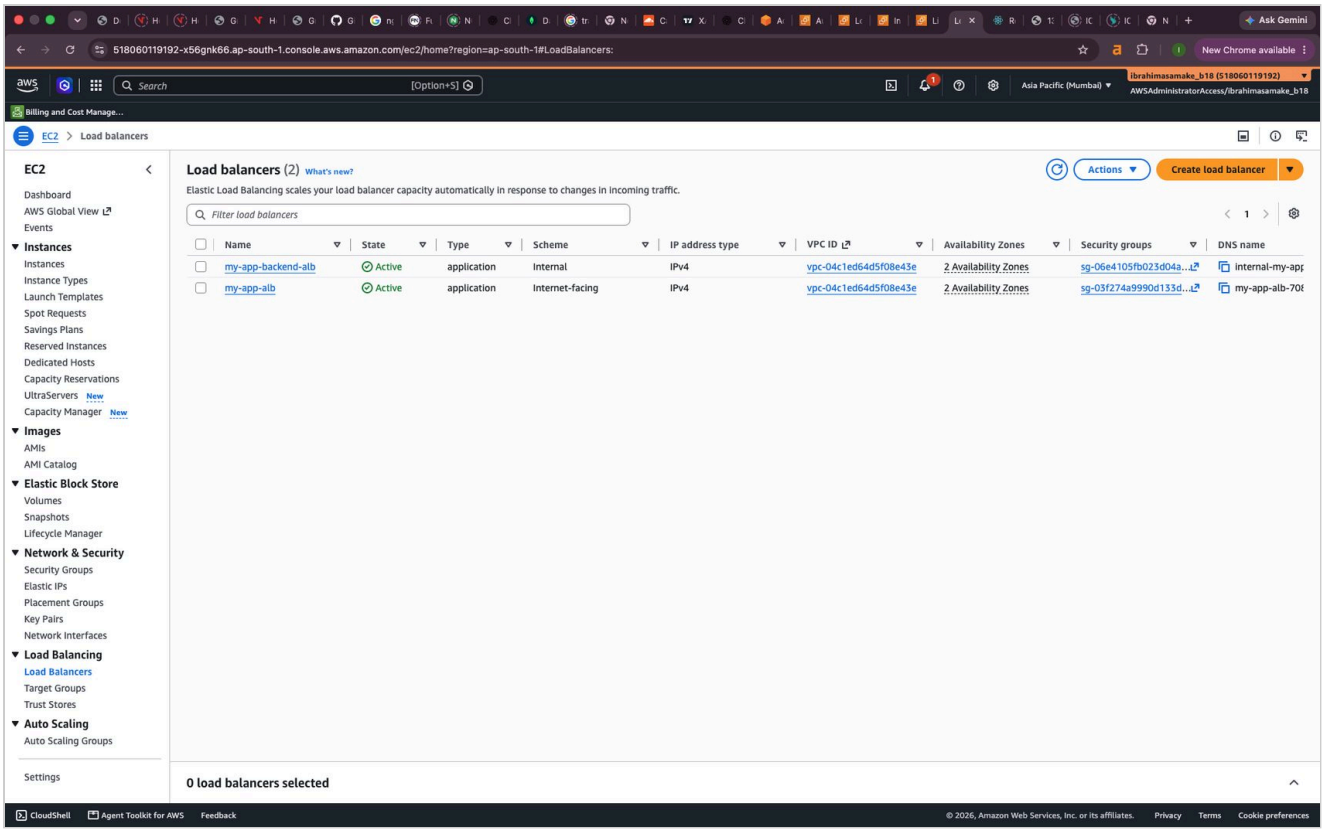


Figure 21 – Load balancers: internal my-app-backend-alb and internet-facing my-app-alb – both Active

6.6 Auto Scaling groups and instances

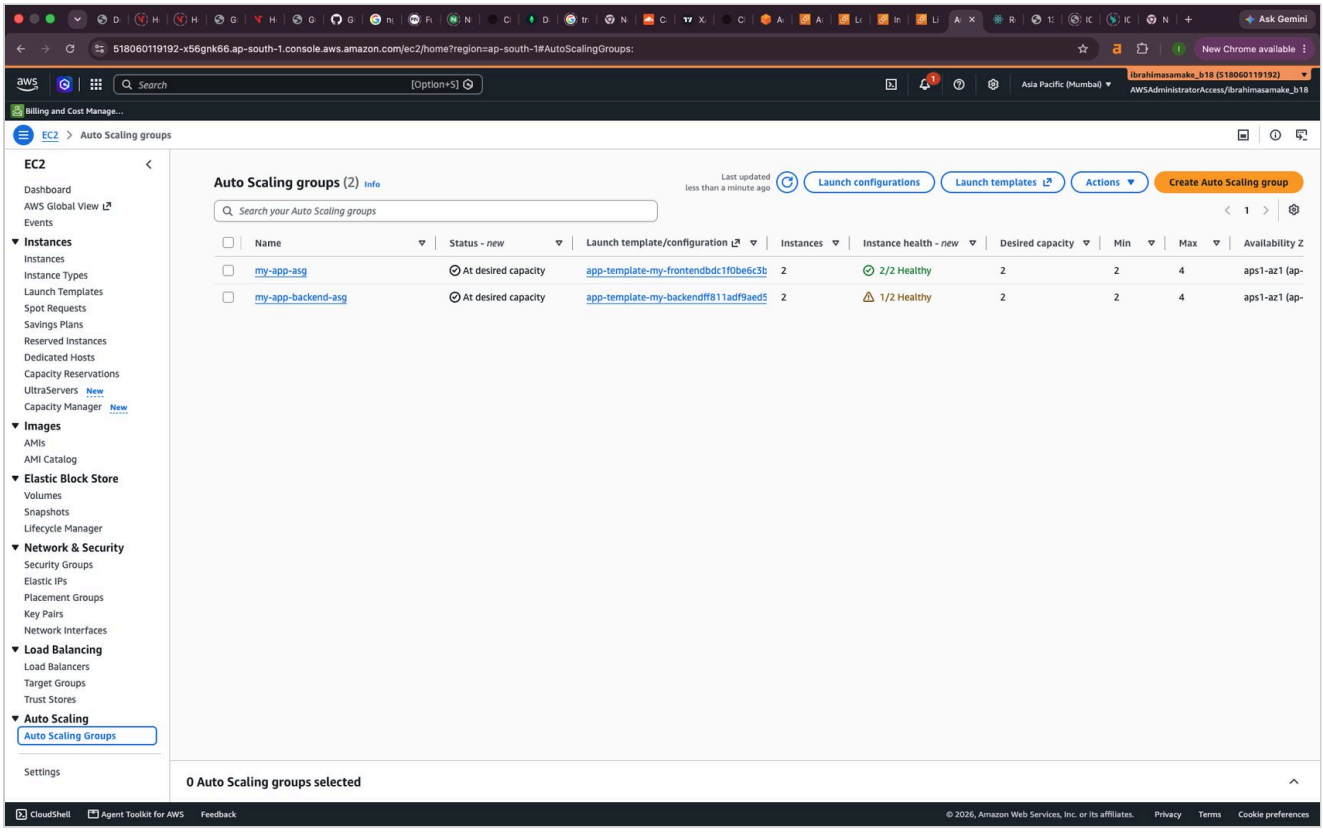


Figure 22 – Terraform-managed ASGs my-app-asg and my-app-backend-asg (desired 2, min 2, max 4)

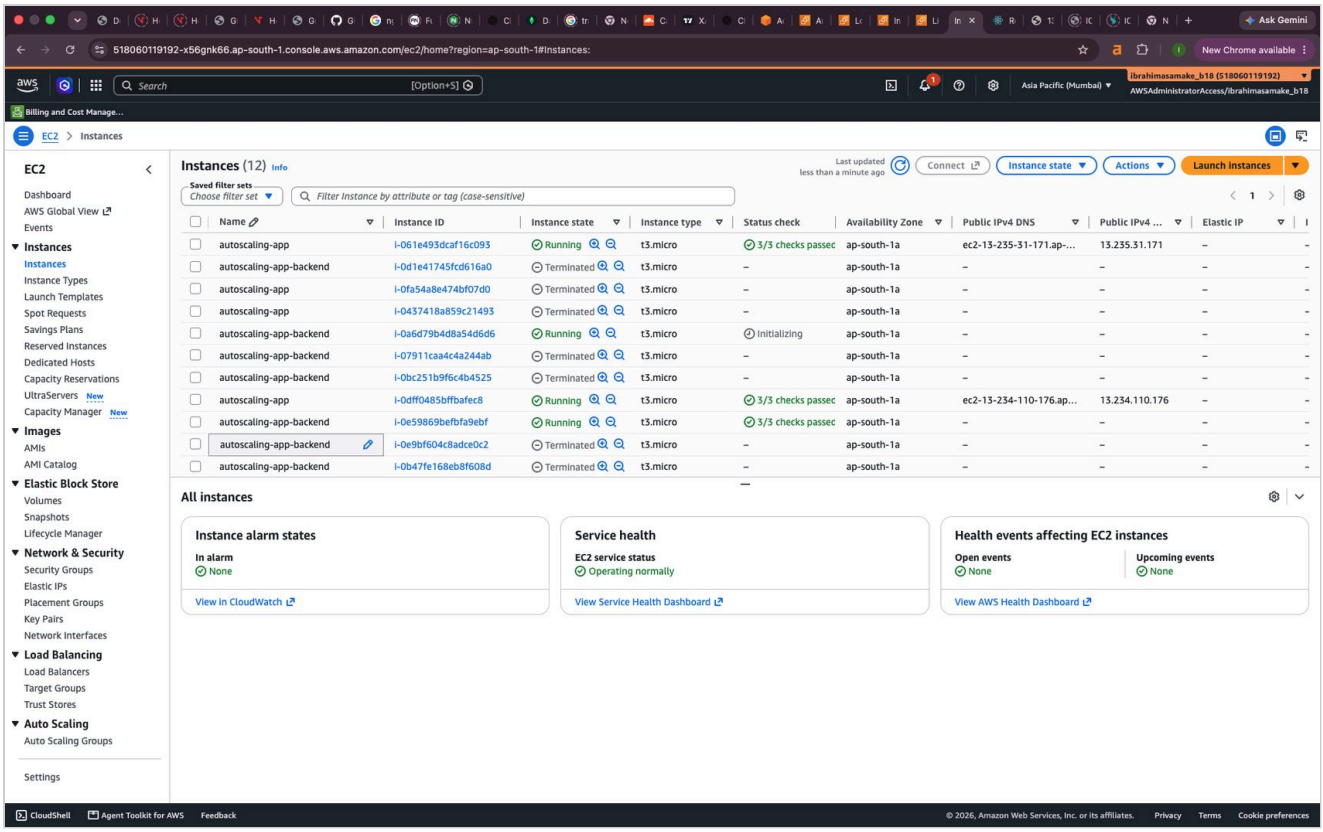


Figure 23 – EC2 instances launched by the ASGs (frontend has public IPs, backend is private). Terminated rows are earlier instances that the ASG replaced.

6.7 Connecting frontend and backend through the load balancers

Because instances now come and go, no file may contain an instance IP. Two settings were changed:

- **frontend/src/url.js** points to the public ALB, so the browser always calls the load balancer.
- **Nginx on the frontend** proxies /trip to the DNS name of the **internal** backend ALB, not to a single backend IP.

```
// frontend/src/url.js
export const baseUrl = process.env.REACT_APP_BACKEND_URL || "http://my-app-alb-708304720.ap-south-1.elb.amazonaws.com";
```

```
location /trip {
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_pass http://internal-my-app-backend-alb-730574711.ap-south-1.elb.amazonaws.com:3001;
}
```

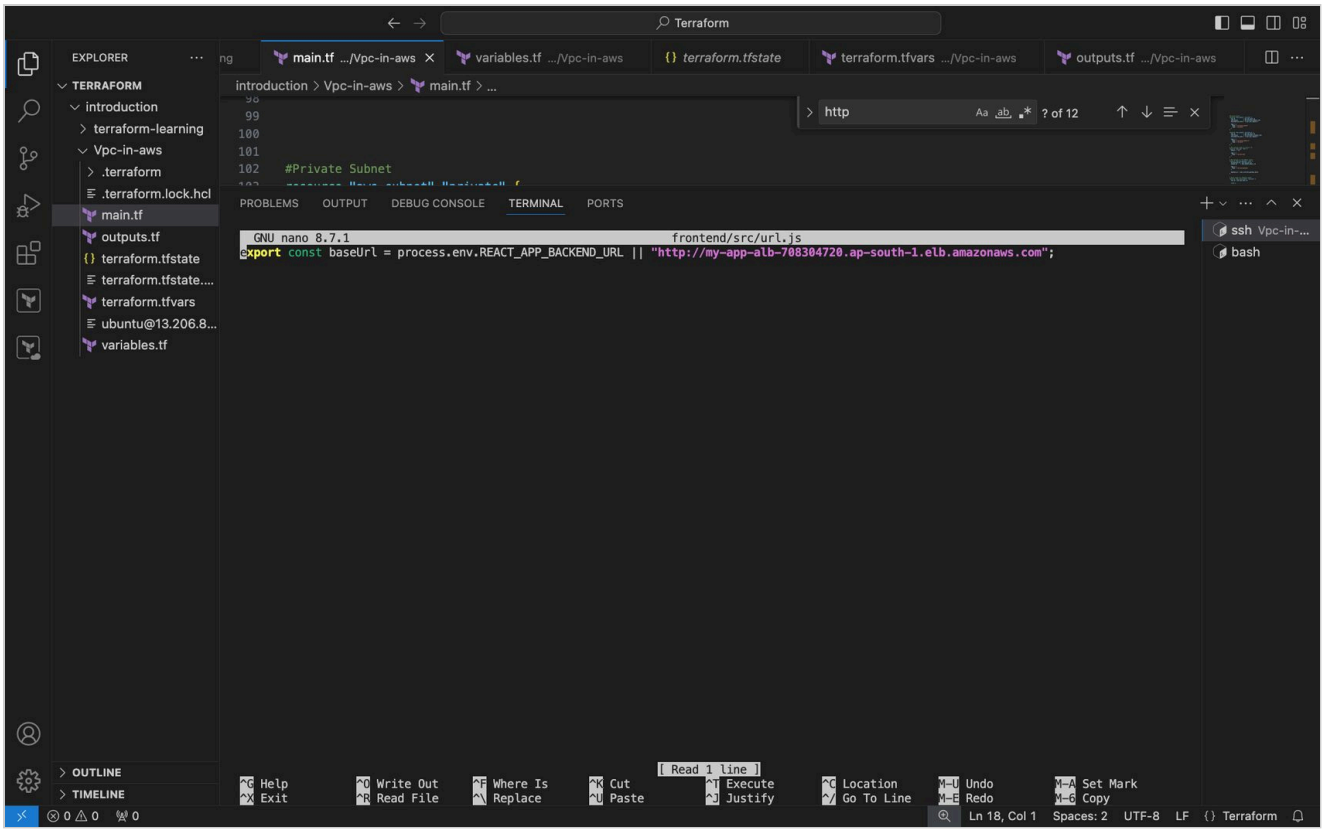


Figure 24 – url.js – baseUrl set to the public load balancer

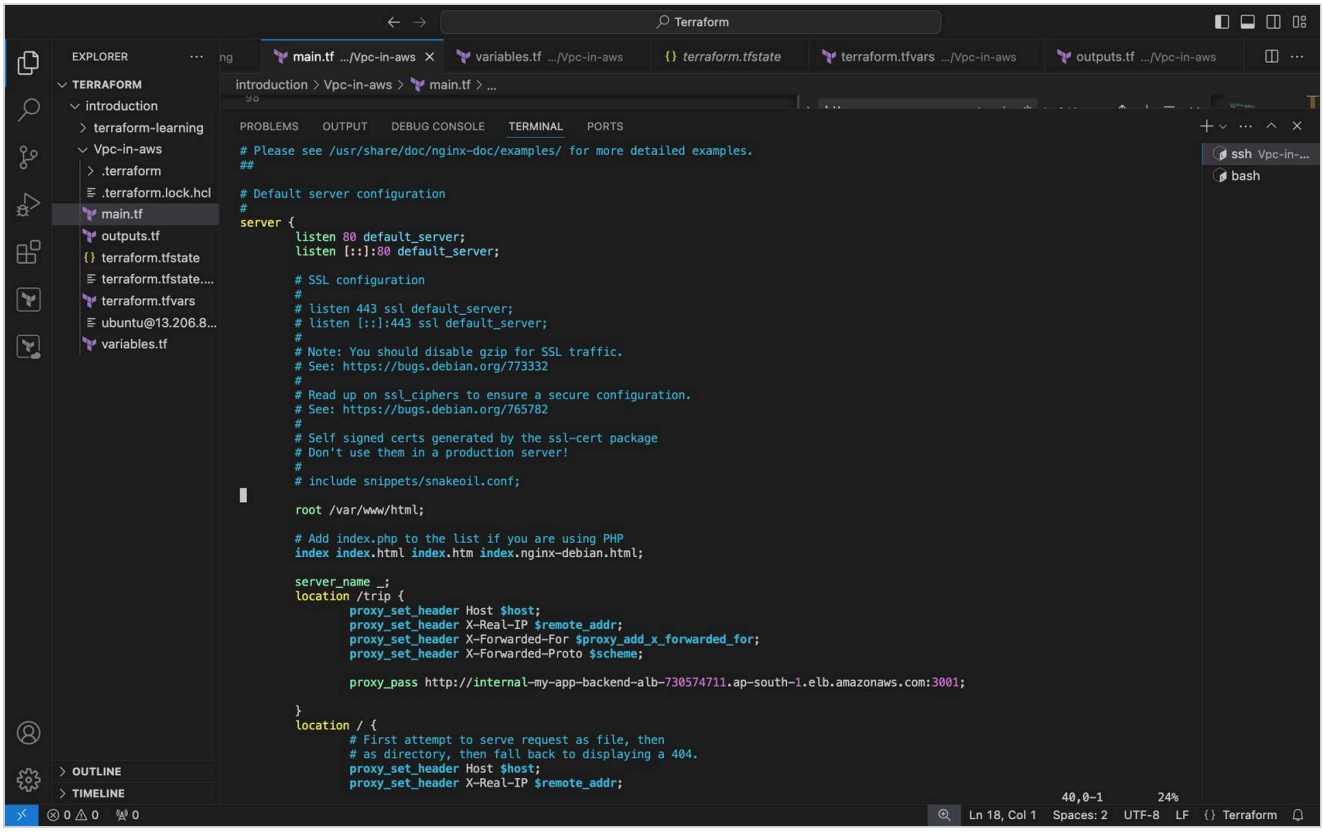


Figure 25 – Nginx – /trip proxied to the internal backend ALB on port 3001

6.8 Verification through the load balancer

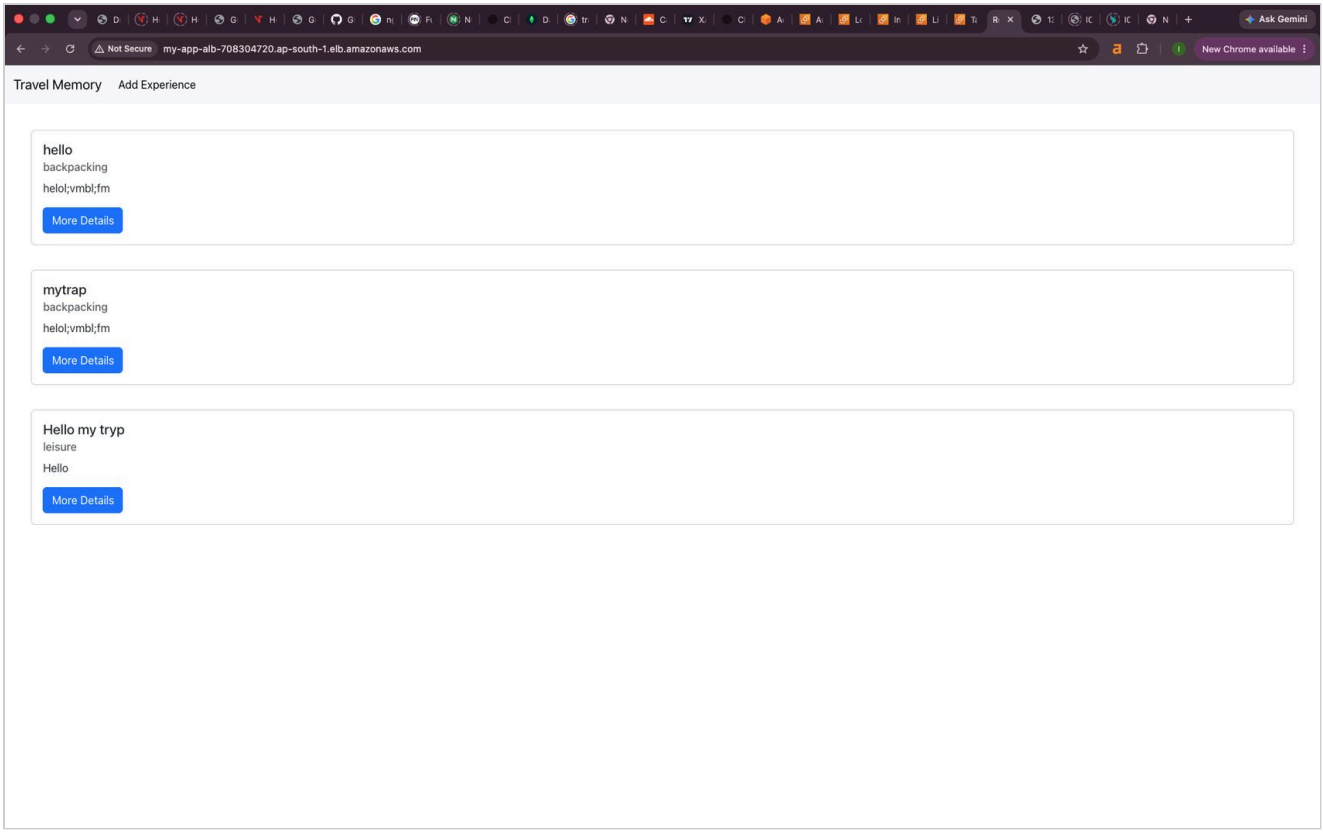


Figure 26 – Application served through the ALB DNS name with records loaded from the backend

7. Custom domain with Cloudflare (Task 4)

7.1 Request an ACM certificate

In AWS Certificate Manager (same region as the ALB), a public certificate was requested for **travelmemory.samibrahima.tech** with DNS validation. ACM provides a CNAME validation record, which was added in Cloudflare (the two _xxxx CNAME records in Figure 29, set to DNS only). The certificate then changed to **Issued**.

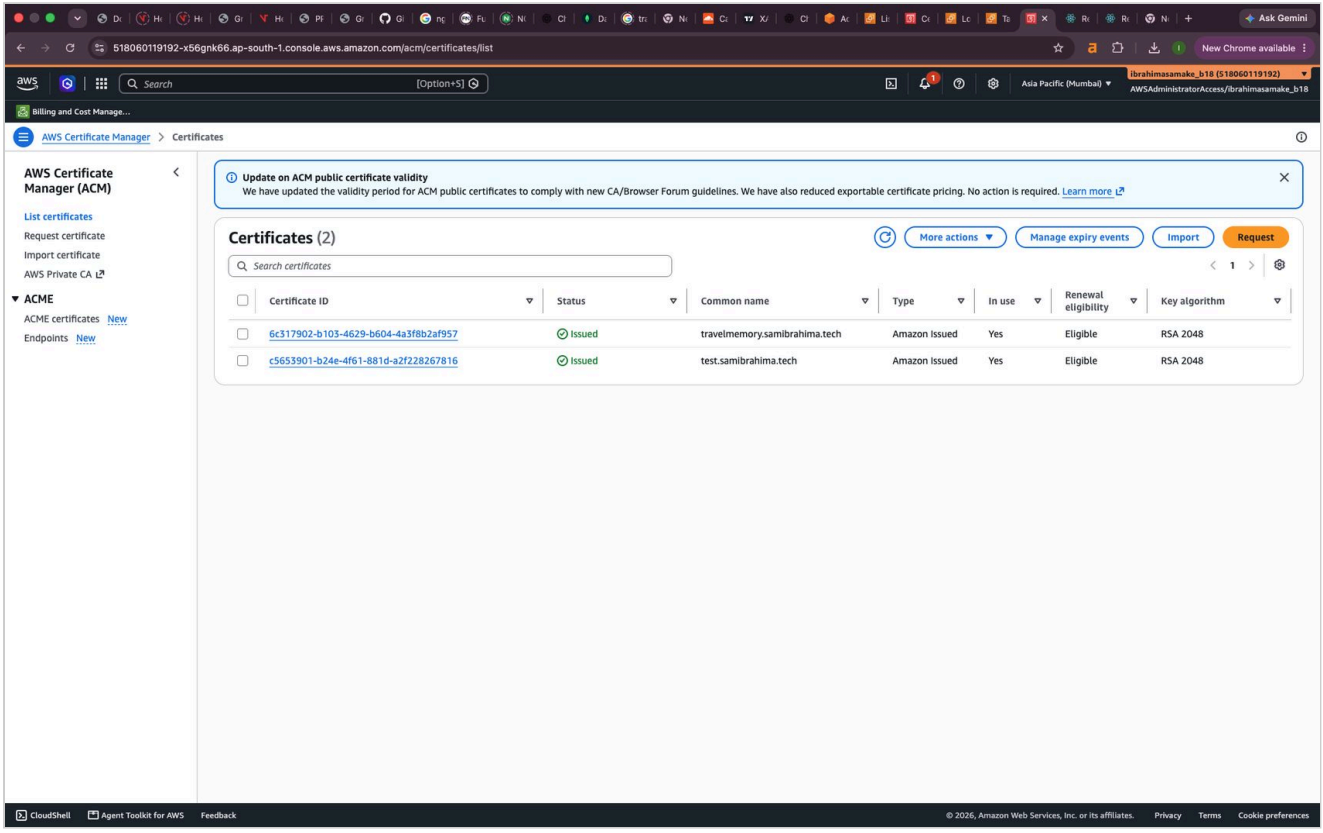


Figure 27 – ACM – certificates for travelmemory.samibrahima.tech and test.samibrahima.tech: Issued, In use

7.2 Add an HTTPS listener to the ALB

An **HTTPS:443** listener was added to my-app-alb using the ACM certificate and the ELBSecurityPolicy-TLS13-1-2 policy, forwarding to app-target-group. The HTTP:80 listener stays in place for plain-HTTP access.

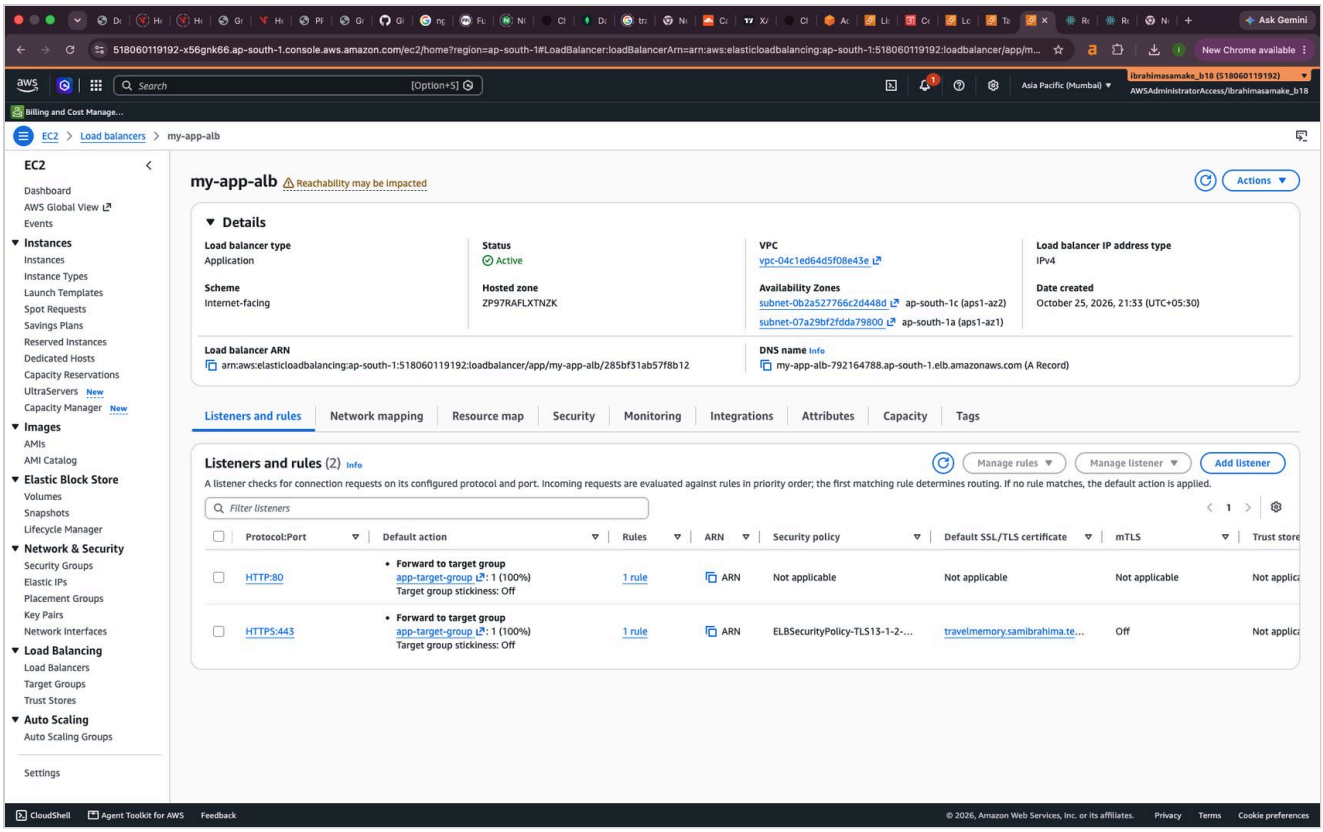


Figure 28 – my-app-alb – listeners HTTP:80 and HTTPS:443 (certificate travelmemory.samibrahima.tech)

7.3 CNAME record → load balancer

In Cloudflare → samibrahima.tech → DNS → Records, a **CNAME** record was created:

Type	Name	Target	Proxy status	TTL
CNAME	travelmemory	my-app-alb-792164788.ap-south-1.elb.amazonaws.com	DNS only	Auto

The CNAME is used because an ALB has no fixed IP address. Its IPs change over time, but its DNS name stays the same.

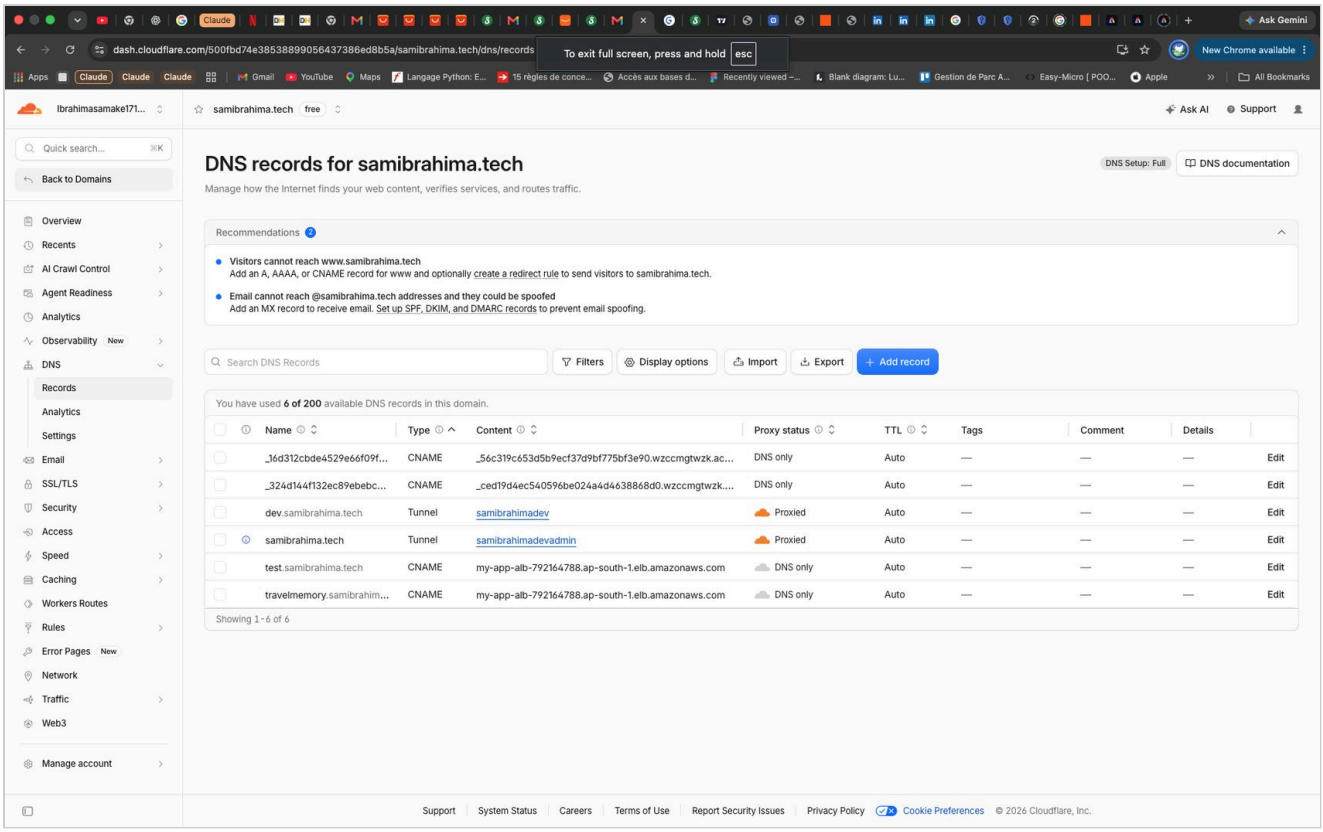


Figure 29 – Cloudflare DNS – travelmemory CNAME → my-app-alb, plus the ACM validation CNAMEs

7.4 A record → frontend EC2 instance

As the brief requires, an **A** record maps a hostname directly to the public IPv4 address of an EC2 instance hosting the frontend:

Type	Name	IPv4 address	Proxy status	TTL
A	frontend	13.235.31.171 (public IP of a frontend instance)	DNS only	Auto

Note: instances in an Auto Scaling group get a new IP whenever they are replaced. For a stable A record, attach an Elastic IP to the instance (or use a dedicated instance). User traffic should always go through the CNAME and the load balancer.

Add screenshot: Cloudflare DNS list showing the A record (e.g. frontend.samibrahima.tech → frontend EC2 public IP).

7.5 Point the frontend at the domain

Finally, url.js was set to the custom domain over HTTPS. This avoids mixed-content errors and keeps the code independent of the ALB name, which changed when the load balancer was recreated (708304720 → 792164788).

```
// /home/ubuntu/TravelMemory/frontend/src/url.js
export const baseUrl = process.env.REACT_APP_BACKEND_URL || "https://travelmemory.samibrahima.tec
h";
```

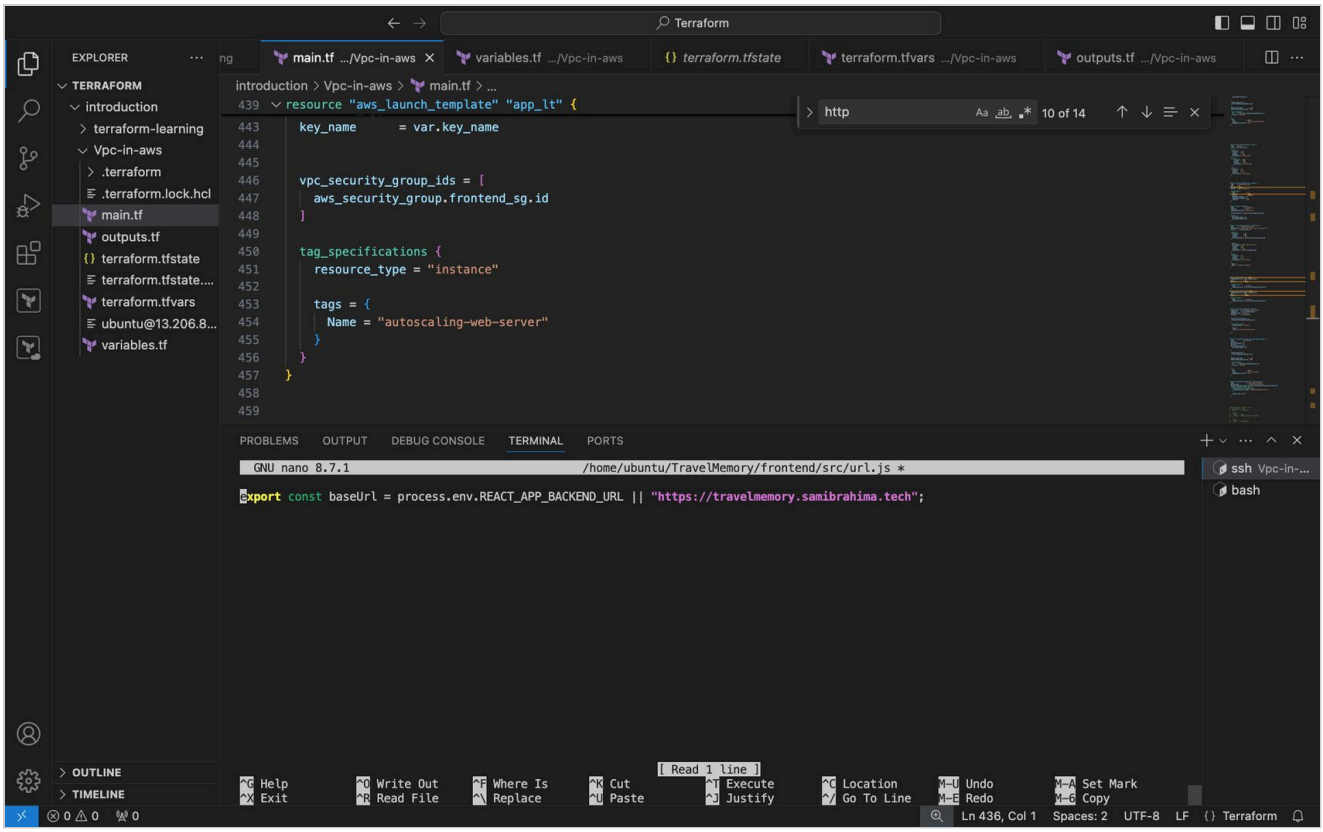


Figure 30 – url.js – baseUrl = https://travelmemory.samibrahima.tech; Terraform launch template in the editor

7.6 Final verification

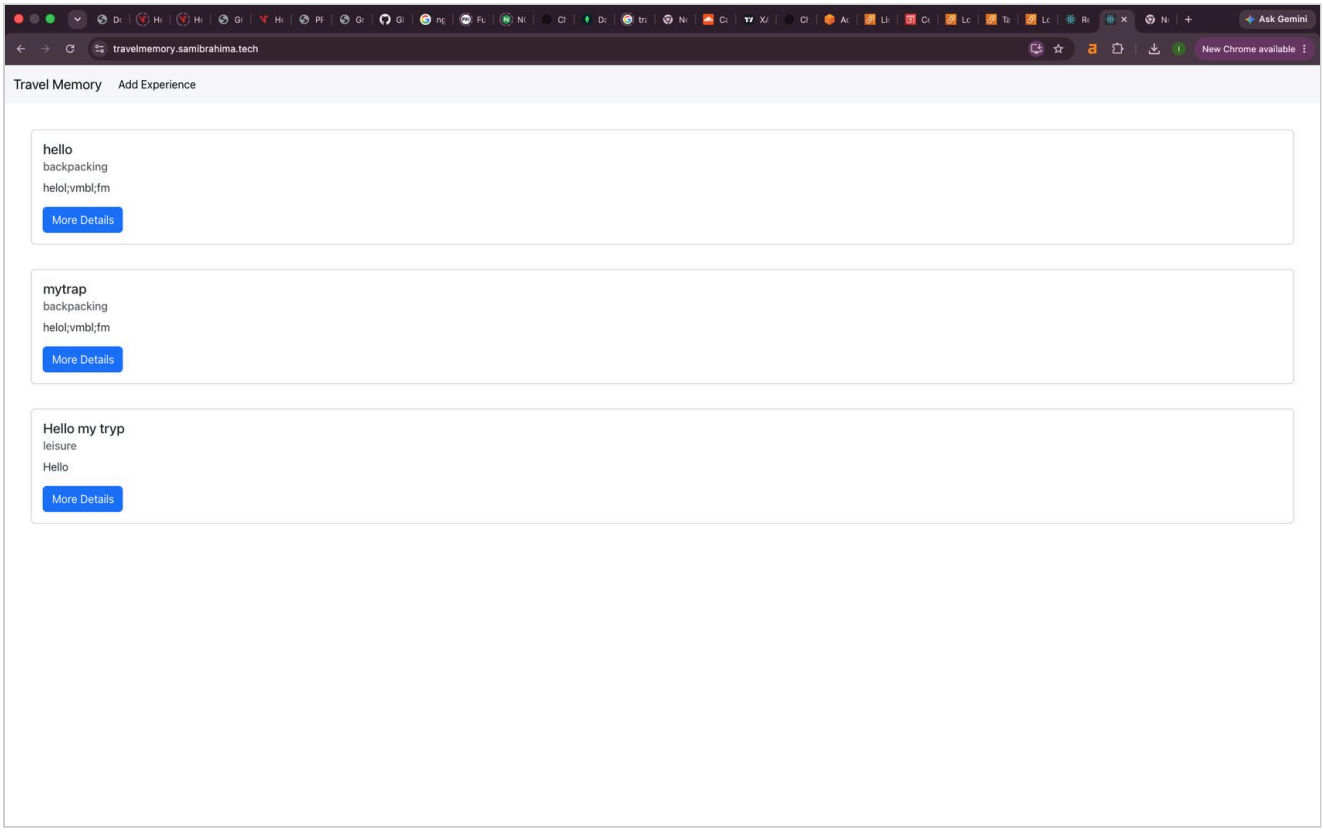


Figure 31 – Travel Memory served on the custom domain travelmemory.samibrahima.tech

8. Security, scalability and resilience

Area	Practice applied
Network isolation	Backend instances sit in private subnets with no public IP. They are reachable only through the internal ALB. Egress goes through the NAT Gateway.
Security groups	Layered rules: ALB allows 80/443 from the internet; frontend allows 80 from the ALB security group (SSH only from the admin IP); internal ALB allows 3001 from the frontend group; backend allows 3001 from the internal ALB group.
Encryption in transit	HTTPS on the ALB with an ACM certificate and a TLS 1.3 / 1.2 security policy; MongoDB Atlas connections use TLS.
Secrets	The database credentials live only in backend/.env on the server. .env is never committed to Git.
Scalability	Two Auto Scaling groups (min 2, max 4) behind load balancers. New instances start from AMIs and launch templates with no manual steps.
Resilience	ALBs span two Availability Zones; health checks replace unhealthy instances automatically; pm2 restarts crashed processes and survives reboots.
Reproducibility	All infrastructure is defined in Terraform (VPC, subnets, gateways, ALBs, launch templates, ASGs), so the environment can be recreated or destroyed with one command.
Loose coupling	No instance IPs in application code: the frontend calls the domain, and Nginx calls the internal ALB DNS name.

Further improvements

- Serve a production build of React (npm run build) directly from Nginx instead of running the development server under pm2.
- Redirect HTTP:80 to HTTPS:443 on the ALB, and turn on Cloudflare proxying with Full (strict) SSL mode.
- Add target-tracking scaling policies (for example, average CPU at 60%) and CloudWatch alarms.
- Store MONGO_URI in AWS Systems Manager Parameter Store or Secrets Manager, and let instances read it through an IAM role.
- Keep the Terraform state in a remote S3 backend with locking.

9. Issues met and how they were solved

Problem	Cause	Fix
pm2: Script not found …/backend/server.js	The repository's entry point is index.js	pm2 start index.js --name my-backend
my-backend 'errored' after 16 unstable restarts (MODULE_NOT_FOUND)	Dependencies were not installed yet	Ran npm install in backend/, then restarted pm2
Backend port clash with React	The React dev server and the API both default to 3000	Set PORT=3001 in .env and proxy /trip to :3001
Frontend broke after the ALB was recreated	url.js still held the old ALB DNS name	Pointed url.js at the custom domain, so a new ALB only needs a CNAME update
Mixed content / insecure warning on HTTP	The site was served over HTTP only	ACM certificate plus an HTTPS:443 listener; baseUrl uses https://

10. Command reference

```
# --- Backend instance ---
git clone https://github.com/UnpredictablePrashant/TravelMemory.git
cd TravelMemory/backend && npm install
nano .env                       # MONGO_URI=..., PORT=3001
sudo npm install -g pm2
pm2 start index.js --name my-backend && pm2 save && pm2 startup

# --- Frontend instance ---
cd ~/TravelMemory/frontend && npm install
nano src/url.js                 # baseUrl = "https://travelmemory.samibrahima.tech"
pm2 start npm --name frontend -- start && pm2 save
sudo nano /etc/nginx/sites-available/default
sudo nginx -t && sudo systemctl restart nginx

# --- Infrastructure ---
terraform init && terraform plan -out tfplan && terraform apply tfplan
terraform output
terraform destroy               # clean-up after grading
```

11. Conclusion and submission

The Travel Memory MERN application runs on Amazon EC2 behind an internet-facing Application Load Balancer and is served over HTTPS at **https://travelmemory.samibrahima.tech**. The frontend and backend each run in an Auto Scaling group of at least two instances. They talk to each other through Nginx and an internal load balancer, the backend stays in private subnets, and the whole environment is defined in Terraform.

Deliverables

- Working application: https://travelmemory.samibrahima.tech
- This document: TravelMemory-Deployment-Documentation.docx
- Architecture diagram: TravelMemory-Architecture.drawio (draw.io source) and TravelMemory-Architecture.png
- GitHub repository (code + Terraform + this document): https://github.com/ibrahimasamake/travel-memory-aws-deployment